Hochschule für Technik Stuttgart
University of Applied Sciences

Master of Science Programme

Photogrammetry and Geoinformatics

Master Thesis

Winter Term 2025/2026

"A Processing and Visualization Workflow for IFC-Based BIM Models Towards Web-Based Digital Twin Applications"

by

Jorge David Velázquez Cupido

Supervisors: Prof. Dr. Angela Blanco-Vogt

M. Sc. Juan Sardi Bazarllo

Declaration

A Processing and Visualization Workflow for IFC-Based BIM Models Towards Web-Based Digital Twin Applications by

Jorge David Velazquez Cupido

A dissertation presented in partial fulfillment of the requirements for the degree of Master of Science in the Department of Geomatics, Computer Science and Mathematics, Stuttgart University of Applied Sciences

I. Statutory declaration

I hereby declare that I have written this thesis independently and have not used any sources or aids other than those indicated. All passages in this thesis that have been taken verbatim or in substance from publications or other external sources are identified as such. This thesis, in the same or a similar version, has not yet been part of any other study or examination.

II. Clarification regarding the use of AI tools

☐ In the creation of this work, AI tools have been used.
☐ No AI tools have been used in the creation of this work.

I hereby confirm that I have informed myself about the possibilities and limitations of the generative AI tools I used. I have made sure that the results provided by the AI tools are correct, or that I have corrected them. I acknowledge that the responsibility for the work lies with the author, not with the AI tools or any other person or authority.

I have only used documented tools (see below) in ways that are according to university guidelines. I take full responsibility for the selection, adoption, and all results of the AI-generated output I have used. This document contains a list of the tools that were used, explicitly naming the tools' names and how these tools were used in the creation of this work.

I hereby grant to Stuttgart Technical University of Applied Sciences permission to reproduce and to distribute publicly paper and electronic copies of this document in whole and in part.

Stuttgart, 26 Feb. 2026 Jorge David Velazquez Cupido

Approved by:

Prof. Dr. Angela Blanco-Vogt

Acknowledgement

First and foremost, I would like to thank God for giving me the grace, strength, and opportunity to complete my Master's studies at the Hochschule für Technik Stuttgart. This achievement, which has represented a great challenge and an enormous personal effort, would not have been possible without the fundamental pillar of my life: my family. To my father, Job Diego Velázquez Moreno; my mother, María Guadalupe Cupido Sánchez; and my brother, Pablo Job Velázquez Cupido, endless thanks. Your unconditional love and support were the driving force that pushed me to never give up and to reach this goal. Likewise, I would like to extend my gratitude to my extended family, especially my uncle Pepe, my uncle Jamie, and Astrid, for their love and companionship along this journey.

I want to express my deepest gratitude to my supervisors, Prof. Dr. Angela Blanco-Vogt and M. Sc. Juan Sardi Barzallo. I thank them immensely not only for pointing me in the right direction but also for their invaluable technical guidance throughout the development of this work; without their patience and constant willingness to guide me, this project would not have materialized.

Finally, I want to thank my friends, who gave me their unconditional backing throughout this journey with their messages of love and encouragement. I make a very special mention of PNDO, BIFC, JAAR, EJMG, MF, GM, XMGS, and LMLO, as well as the great friendships I forged during my time at the university. To my dearest "lindavos" friends, with a very special thanks to Daniel and Rodrigo, thank you for always being there. A special thanks also goes to the German friends I met playing volleyball; thank you for welcoming me, including me in your plans, and making me feel at home. To all the other friends, colleagues, and special people who accompanied me from a distance: thank you for believing in me and being part of this journey. I am also grateful to this city, Stuttgart, and the experiences lived here, which have shaped me not only as a professional but also as a person.

"When a dream dies, it is because it has come true."

Stuttgart, February 2026

Jorge David Velázquez Cupido

Contents

Declaration 2
Acknowledgement 3
Contents 4
Abstract 9
List of Tables 11
List of Figures 12
Abbreviations 13
1. Introduction 14
1.2 Problem Statement 14
1.3 Objective 14
1.3.1 General objective 14
1.3.2. Specific objectives 15
1.4 Research questions 15
1.5 Scope and limitations 15
1.6 Structure of the thesis 16
2. Background 17
2.1 Facility Management and Information Needs 17
2.2 Building Information Modelling (BIM) and Industry Foundation Classes (IFC) for Facility Management (FM) 17
2.3 BIM – GIS integration and georeferencing of Buildings 18
2.4 Internet of Things (IoT) and sensor systems in buildings 19
2.5 Web – based 3D visualization and Digital Twin concepts 20
3. Related Work and Existing Frameworks 21
3.1 BIM – based FM tools and CAFM/CMMS systems 21
3.2 Research on BIM – FM integration using IFC 22
3.3 BIM – GIS and city – scale integration approaches 22
3.4 BIM – IoT / Building Digital Twin Frameworks 23
3.5 Open – Source ecosystem for IFC on the web 23
3.6 Summary of gaps and requirements for the thesis 24
4. Methodology and Research Design 26
4.1 Research approach 26
4.2 Problem framing and scope 26
4.3 Requirements engineering 26

4.4 Development Methodology 26
4.5 Case study and dataset 27
4.6 System integration workflow (end to end) 27
4.7 Evaluation Strategy 27
4.8 Data Governance, Ethics, and Constraints 27
4.9 Summary of methodology 28
5 Framework Design and System Architecture 29
5.1 Requirements for an open-source FM framework 30
5.2 Conceptual data model 32
5.2.1 IFC as semantic backbone (GlobalId, spaces, systems) 33
5.2.2 FM entities: notes, work orders, assets, sensors 33
5.2.3 Hybrid persistence architecture 34
5.3 Overall system architecture (BIM + GIS + IoT) 34
5.3.1 Presentation layer (client) 34
5.3.2 Application Layer (API) 35
5.3.3. Data Layer (storage) 35
5.4 Integration strategies 35
5.4.1 IFC ↔ GIS 35
5.4.2 IFC ↔ sensors 36
5.5 Security, data ownership and open-source considerations 36
6 Implementation 37
6.1 Backend implementation (FastAPI + IfcOpenShell) 37
6.1.1 Project structure and main components 37
6.1.2 IFC services (loading, property queries, spatial structure) 38
6.1.3 GIS services (footprint, campus layers) 38
6.1.4 FM data models and database schema 38
6.1.5 REST and streaming endpoints 39
6.2. Frontend implementation (Vite + web-ifc + three.js +Cesium) 39
6.2.1 IFC 3D viewer and properties panel 39
6.2.2 Georeferencing status and calibration helper 40
6.2.3 Category filters and view modes 40
6.2.4 FM tools 40
6.2.5 Mini BIM/FM editor 40
6.2.6 Sensor visualization and (near) real-time updates 41

6.3 Deployment and open-source repository 41
7 Case Study and Evaluation 42
7.1 Evaluation setup 42
7.2 Defined Facility Management (FM) Use-Cases 42
7.2.1. Model Management & Spatial Context 43
7.2.2 Geospatial Alignment & Calibration 44
7.2.3 Facility Operations & Reporting 45
7.2.4 Real-Time IoT Monitoring 47
7.2.5 Advanced BIM Coordination 47
7.2.6 Interactive Inspection & Modification 50
7.3 Functional evaluation 52
7.4 Technical observations 54
7.5 Discussion 54
8 Conclusions and Future Work 56
8.1 Summary of contributions 56
8.2 Answers to research questions 57
8.2.1 Research Question 1 (RQ1): BIM/FM Data Linking 58
8.2.2 Research Question 2 (RQ2): BIM-GIS Integration and Placement 58
8.2.3 Research Question 3 (RQ3): IoT Integration in an FM Prototype 59
8.2.4 Research Question 4 (RQ4): Validation and Feasibility 59
8.3 Implications for Facility Management practice 59
8.4 Limitations of the Study 60
8.5 Future work 60
8.5.1 Robust BIM Authoring 60
8.5.2 AI-Driven Operational Intelligence 61
8.5.3 Advanced GIS & Semantic Context 61
8.5.4 Immersion and Field Integration (WebXR) 61
8.5.5 Industry Standardization (BCF & ISO) 61
8.5.6 Regulatory Audit and Automatic Compliance 61
8.5.7 Operational Intelligence and Graph Data Fusion 61
8.5.8 Lifecycle and Collaboration (4D/5D) 62
9. Summary 63
Overview & Problem Statement 63
Proposed Solution 63

Core Architecture & Technical Features 63
- IFC Semantic Backbone 63
- Hybrid BIM-GIS Visualization 63
- Real-Time IoT Integration 63
- Decoupled Backend 63

Results & Conclusion 63
Appendixes 66
Appendix 1 User Workflows & Operational Procedures 66
Workflow 1: Model Loading & Management 66
Workflow 2: Model Tree Navigation 66
Workflow 3: 2D Plan Navigation 67
Workflow 4: Measurements & Dimensioning 67
Workflow 5: Calibration & Georeferencing 68
Workflow 6: BIM Editing (Transformations) 69
Workflow 7: Asset Management (New!) 69
Workflow 8: Systems & Disciplines (Tech Mode) 70
Workflow 9: Clash Detection 70
Workflow 10: Model Comparison (Diff) 71
Workflow 11: IoT Sensor Monitoring 71
Workflow 12: Maintenance Work Orders 72
Workflow 13: Reports (QTO) 72
Workflow 14: Section Box & Clipping 72
Workflow 15: ViewCube Navigation 73
Workflow 16: GIS Context (Surroundings) 73
Workflow 17: Box Selection 74
Workflow 18: Advanced Search & Filtering (Explorer) 74
Workflow 19: Element Annotations (Notes) 75
Workflow 20: Language Switching 75
Workflow 21: Visualization Control 76
Workflow 22: GIM Layer (Geo-Footprint) 76
Workflow 23: Project Creation (Upload) 77
Appendix 2 Project Architecture 77
Project Architecture 77
Component Diagrams 79

Critical Data Flows 80
API Endpoints (FastAPI) 80
Overview of Tools Used 84
References 86

Abstract

Current Facility Management (FM) workflows are frequently hindered by proprietary platforms that restrict data reuse and long-term ownership. Furthermore, critical operational information remains fragmented across isolated domains: IFC-based models, geospatial contexts, and dynamic IoT sensor streams. To bridge this gap, this research presents a modular, web-based, open-source framework that unifies these heterogeneous sources into a working prototype (MVP) that supports day-to-day FM tasks, such as asset lookup, issue reporting, work-order tracking, and real-time sensor monitoring.

The proposed architecture establishes a semantic backbone based on IFC, meaning that element types, relationships, and GlobalId (GUID) identifiers provide the shared reference structure used to connect FM entities and sensor data to specific BIM components. This IFC layer is processed server-side via IfcOpenShell, enabling reliable querying, enrichment, and updates while maintaining standards compliance. By relying on these stable identifiers and open schemas, the approach avoids vendor lock-in and preserves long-term data interoperability. The backend exposes a flexible REST (Representational State Transfer) API that follows hypermedia/HATEOAS principles, meaning that responses include links to related resources and allowed actions, so that Create, Read, Update, and Delete (CRUD) operations remain explicit, discoverable, and traceable (Herlé & Blankenbach, 2024).

On the client side, the framework successfully integrates BIM and GIS by combining Cesium for geospatial context and Three.js for detailed model rendering, enabling interactive navigation, element selection, and attribute querying directly in the browser. This hybrid visualization is accelerated through WebAssembly (WASM) parsing, supporting responsive interaction even with large IFC models. In addition, the system integrates an IoT layer that streams sensor observations in real time and links them to BIM components via GlobalId (GUID), enabling environmental monitoring (e.g., CO_2 sensors and thermal comfort indicators) within the same web interface.

Validated through a campus-based case study and guided by a Design Science Research (DSR) methodology, the framework was designed, implemented, demonstrated, and evaluated as a reproducible web-based prototype. The results show that integrating BIM, GIS, and IoT sensor streams through open standards is feasible within a single browser-based interface, providing a practical open-source alternative to proprietary digital-twin authoring workflows.

Key words: IFC, BIM, GIS, IoT, GUID, CRUD, REST, API, WASM, GlobalID, Digital Twins, FM, FastAPI, Open-Source, DSR, IfcOpenShell, CesiumJS, Three.js.

List of Tables

Table 1 Functional 32
Table 2 Architecture Contributions 57
Table 3 Project Architecture 79
Table 4 Frontend Architecture 79
Table 5 Backend Architecture 80

List of Figures

Figure 1 Architecture 29
Figure 2 Spatial hierarchy 33
Figure 3 Project Administration (Tool 22) Internationalization module (Tool 23) 43
Figure 4 the Structure Tree (Tool 9) 44
Figure 5 Asset Management (Tool 14) 44
Figure 6 Georeferencing (Tool 21) 45
Figure 7 The Global 3D Environment (Tool 13) 45
Figure 8 Annotations (Tool 17) 46
Figure 9 Facility Management (Tool 8) 46
Figure 10 IoT Module (Tool 4) 47
Figure 11 Section Box (Tool 15) 48
Figure 12 Section Box (Tool 15) 48
Figure 13 Floor Plans tool (Tool 6) 49
Figure 14 The Systems Analysis / Tech Mode (Tool 7) 49
Figure 15 Advanced Coordination (Tool 5) 50
Figure 16 the BIM Editor (Tool 3) 51
Figure 17 Selection & Properties (Tool 10) 51
Figure 18 Advanced Search (Tool 19) 52
Figure 19 Summary 65

Abbreviations

AABB	Axis-Aligned Bounding Box
AI	Artificial Intelligence
API	Application Programming Interface
BCF	BIM Collaboration Format
BIM	Building Information Modelling
BVH	Bounding Volume Hierarchies
CAFM	Computer-Aided Facility Management
CMMS	Computerized Maintenance Management Systems
CO_2	Carbon Dioxide
COBie	Construction Operations Building Information Exchange
CPS	Cyber-Physical Systems
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DSR	Design Science Research
ECEF	Earth-Centered, Earth-Fixed
FM	Facility Management
GIS	Geographic Information Systems
GLB	GL Transmission Format Binary
GUID	Globally Unique Identifier
HATEOAS	Hypermedia as the Engine of Application State
HTTP/HTTPS	Hypertext Transfer Protocol / Secure
HVAC	Heating, Ventilation, and Air Conditioning
IEQ	Indoor Environmental Quality
IFC	Industry Foundation Classes
IoT	Internet of Things
ISO	International Organization for Standardization
IWMS	Integrated Workplace Management Systems
JSON	JavaScript Object Notation
LoD	Level of Detail
LoI	Level of Information
MEP	Mechanical, Electrical, and Plumbing
MQTT	Message Queuing Telemetry Transport
MVP	Minimum Viable Product
O&M	Operation and Maintenance
OSM	OpenStreetMap
QTO	Quantity Takeoff
REST	Representational State Transfer
RQ	Research Question
SSE	Server-Sent Events
WASM	WebAssembly
WebGL	Web Graphics Library
WebXR	Web Extended Reality
WGS84	World Geodetic System 1984

1. Introduction

The management of built assets during their Operation and Maintenance (O&M) phase represents the longest and most capital-intensive period of a facility's lifecycle. To optimize these processes, the industry has increasingly adopted Building Information Modeling (BIM) methodologies. However, a significant gap remains between the static, highly detailed geometric data of as-built BIM models and the dynamic, real-time operational needs of Facility Management (FM). While standards like the Industry Foundation Classes (IFC) provide a robust semantic foundation, their practical integration with geographic context (GIS) and live sensor telemetry (IoT) in accessible environments is still an ongoing challenge. This research is motivated by the need to democratize digital twins, moving away from fragmented, proprietary software ecosystems towards cohesive, open-source, web-based architectures that grant asset owners full data sovereignty and operational continuity.

1.2 Problem Statement .

Facility Management (FM) teams depend on multiple digital systems to manage buildings during operation, such as CMMS/CAFM platforms for maintenance processes, BIM models for as-built documentation, GIS for spatial context, and separate IoT/Building Management Systems for sensor monitoring. In practice, these information sources remain fragmented: work orders and issues are often tracked outside the BIM model, sensor data is visualized in isolated dashboards, and the building's location/context is handled in separate GIS tools. This fragmentation leads to duplicated data entry, weak traceability between operational records and building components, and limited reuse of BIM deliverables for daily FM decision-making.

Although open standards and open-source components exist (e.g., IFC for building semantics, web-based 3D visualization, and lightweight APIs for data exchange), a practical gap remains: there is a lack of a coherent, web-based, open-source framework that integrates BIM, GIS, and IoT in a single workflow while preserving long-term interoperability and data ownership. The key challenge is not only rendering an IFC model in the browser, but enabling reliable semantic linking between FM entities (notes, assets, work orders) and BIM elements, adding geospatial context (including handling imperfect or missing georeferencing), and supporting near real-time sensor overlays through a standards-oriented API architecture. This thesis addresses that gap by designing and validating a modular prototype that demonstrates end-to-end integration for realistic FM tasks directly in the browser.

1.3 Objective

1.3.1 General objective

Develop and validate a practical, open-source, web-based Facility Management (FM) framework that integrates an Industry Foundation Classes (IFC) model with geospatial context (CesiumJS) and live IoT sensor data. The framework leverages IFC semantics and persistent IFC GlobalId (GUID) identifiers to link operational FM records to building elements, enabling day-to-day FM tasks—including Create, Read, Update, and Delete (CRUD) operations—directly in the browser.

1.3.2. Specific objectives

1. Implement a web-based IFC viewer and supporting backend services to load IFC models, browse the spatial hierarchy, perform element selection, and inspect properties, using IFC GlobalId (GUID) as the consistent linking key for all BIM-referenced actions.
2. Integrate BIM with real-world geospatial context by embedding the model in CesiumJS (3D globe/terrain) and providing a lightweight georeferencing status + manual calibration workflow for IFC files with missing or unreliable georeferencing, enabling the model to be placed and explored in its campus-scale surroundings.
3. Enable and evaluate FM interactions through CRUD operations for core FM entities (e.g., assets, notes, work orders, and sensor metadata) and integrate IoT sensor streams (e.g., CO_2 / comfort) through near real-time updates linked to BIM semantics (GlobalId / space-level mapping). Validate the prototype via scenario-based tests representative of FM practice, including responsiveness, IFC loading behavior, and streaming latency.

1.4 Research questions

1. RQ1 — BIM/FM data linking:
   How can an open-source, web-based framework use IFC semantics and persistent IFC GlobalId (GUID) identifiers to reliably link Facility Management (FM) records (e.g., assets, notes, work orders) to specific BIM elements in the browser?
2. RQ2 — BIM–GIS integration and placement:
   How can an IFC model be integrated with geospatial context in CesiumJS, and what lightweight workflow enables practical placement/georeferencing of IFC models when native georeferencing is missing or unreliable?
3. RQ3 — IoT integration in a web-based FM prototype:
   How can real-time IoT sensor streams be integrated into the same web application and associated with BIM components (e.g., via GUID or space-level mapping) to support environmental monitoring and operational decision-making?
4. RQ4 — Validation and feasibility:
   To what extent does the proposed prototype support day-to-day FM tasks through CRUD operations, and how does it perform under realistic usage scenarios in terms of responsiveness, IFC loading behavior, and streaming latency?

1.5 Scope and limitations

This thesis focuses on designing and validating a practical open-source web framework for Facility Management that integrates IFC-based BIM, GIS context in CesiumJS, and real-time IoT sensor data within a single browser-based application. The scope covers: (i) loading and visualizing IFC models, browsing spatial structure, selecting elements, and querying properties; (ii) linking FM entities (assets, notes, work orders, sensor metadata) to BIM elements via IFC GlobalId (GUID); (iii) integrating geospatial context and supporting a lightweight placement/calibration workflow when IFC georeferencing is missing or unreliable; and (iv) validating feasibility through scenario-based tests that reflect typical FM tasks.

The work is not intended as a full BIM authoring environment. Advanced model editing, clash detection, discipline-specific coordination workflows, and detailed IFC schema validation are outside the scope. The prototype prioritizes interoperability and reproducibility over exhaustive

feature completeness; therefore, only a representative subset of FM workflows is implemented. IoT integration is demonstrated through selected environmental sensors and a reference streaming mechanism, and results are evaluated on a campus-based case study rather than across multiple buildings or long-term operational deployments. Performance evaluation is limited to practical indicators (e.g., responsiveness, IFC loading behavior, and streaming latency) and does not constitute a comprehensive benchmark across hardware platforms or competing commercial systems.

Despite these limitations, the thesis demonstrates the feasibility of a modular open-source workflow towards web-based digital twin applications.

1.6 Structure of the thesis

Chapter 1 introduces the research context and motivation, defines the problem statement, and presents the objectives, research questions, and scope of the work.

Chapter 2 provides the background required to understand the thesis, covering Facility Management information needs, BIM and IFC concepts for operations, BIM–GIS integration and georeferencing, IoT/sensor systems in buildings, and web-based 3D visualization and Digital Twin fundamentals. Chapter 3 reviews related work and existing frameworks, including BIM-based FM tools (CAFM/CMMS), IFC-based BIM–FM research, BIM–GIS integration approaches, BIM–IoT and Digital Twin frameworks, and the open-source ecosystem for IFC on the web; it concludes with a summary of gaps and requirements that motivate the proposed approach.

Chapter 4 describes the research design and methodology, following a Design Science Research perspective, and details requirements engineering, development methodology, case study setup, the end-to-end BIM–GIS–IoT integration workflow, evaluation strategy, and relevant constraints (e.g., data governance and ethics).

Chapter 5 presents the framework design and system architecture, including the conceptual data model, overall layered architecture (client, API, and storage), integration strategies between IFC–GIS and IFC–sensors, and open-source/security considerations.

Chapter 6 documents the implementation of the prototype, describing the backend services (FastAPI and IfcOpenShell), frontend components (Vite, web-ifc/WebAssembly, Three.js, and CesiumJS), interaction tools, FM functionality, and sensor visualization.

Chapter 7 reports the case study and evaluation results, defines representative FM use cases, presents functional and technical observations, and discusses limitations and lessons learned. Finally,

Chapter 8 concludes the thesis by summarizing contributions, answering the research questions, outlining implications for Facility Management practice, and proposing directions for future work.

2. Background

This chapter provides the theoretical and technical background required for the thesis. It first introduces Facility Management (FM) and the associated information needs. It then reviews BIM concepts for FM with a focus on interoperability through the Industry Foundation Classes (IFC) standard. Next, the chapter summarizes BIM–GIS integration principles, including georeferencing challenges when aligning local BIM coordinates with global reference frames. Finally, it introduces IoT and sensor systems in buildings and outlines key concepts of web-based 3D visualization and Digital Twins that motivate the proposed open-source framework.

2.1 Facility Management and Information Needs

Facility Management (FM) is a professional discipline of management, focused on coordinating the use of space, infrastructure, people, and organization resources to ensure that physical assets and environments are managed effectively to meet need of their users, by integrating maintained, building services engineering, facilities engineering, safety, efficient, and comfort. It integrates multiple disciplines in order to have an influence on the efficiency and productivity of economies of societies. The benefits of an integrated system standard for FM include: (*ISO 41001:2018(En), Facility Management — Management Systems — Requirements with Guidance for Use*, n.d.)

To improve workforce productivity, safety, health and well-being.
To improve communication.
To improve efficiency and effectiveness
To improve service consistency
To provide a common platform for the organization.

FM Lifecycle and Costs It is crucial to differentiate between design/construction and Operation and Maintenance (O&M). The O&M phase represents the "longest and costliest period" in a facility's lifecycle, meaning operational costs far exceed construction costs. Consequently, having accurate asset data is vital for achieving economies of scale and reducing the total cost of ownership (Pinti et al., 2022a).
There are different types of FM:

- Hard FM focuses on the building fabric and technical systems, such as HVAC, energy, and maintenance.
- Soft FM: Focuses on services for users, such as cleaning, catering, and security.

2.2 Building Information Modelling (BIM) and Industry Foundation Classes (IFC) for Facility Management (FM)

Building Information Modeling (BIM) refers to the digital management and exchange of structured information about built assets across their life cycle, enabling more effective collaboration among stakeholders. In this context, openBIM describes BIM implemented through open standards to address long-standing interoperability and communication challenges (*openBIM*, n.d.)

In Facility Management (FM), BIM helps consolidate both geometric and non-geometric asset information into an information model that can be exchanged through one or multiple information containers (e.g., IFC files), depending on the project scope and purpose

(buildingSMART Spain, 2025). The fundamental role of BIM in this phase is therefore to centralize and structure reliable asset information that supports operations throughout the life cycle.

- BIM supports asset management from planning and construction through operation and maintenance. This is particularly important because the operation and maintenance phase is typically the longest and most costly stage of the lifecycle (Hakimi et al., 2024).
- BIM functions as an information repository that brings together a large amount of structured data to support decision-making in FM activities such as maintenance, energy management, and building operations (Liu et al., 2020).
- BIM is a key foundation for Smart Facility Management and Digital Twins, as it provides the semantic and geometric reference model. When BIM is integrated with sensors and the Internet of Things (IoT), it enables real-time monitoring by linking operational observations to the digital representation of the facility (Hakimi et al., 2024).

The implementation of BIM in the operational phase provides several advantages for Facility Management, with improvements in efficiency, cost, and environmental performance.

In the area of maintenance, BIM supports predictive maintenance: combining Artificial Intelligence (AI) and sensor data can help detect failures earlier and support more cost-effective maintenance planning. BIM also improves asset localization and provides access to relevant history and documentation (e.g., model information and warranties). In addition, 3D visualization facilitates access to and understanding of the information during inspections (Pinti et al., 2022b).

In the area of energy and retrofit planning, BIM enables simulations and operational analyses of the building, supporting renovation and modernization decisions. (Liu et al., 2020)

In terms of cost and information exchange, consolidating information reduces duplication and helps minimize information loss across handover. By using open standards such as IFC (Industry Foundation Classes) and COBie (Construction Operations Building Information Exchange), BIM supports interoperability between design software and FM platforms such as CAFM (Computer-Aided Facility Management) systems, reducing data loss during transfer (Liu et al., 2020).

2.3 BIM – GIS integration and georeferencing of Buildings

The primary motivation for integrating Building Information Modelling (BIM) and Geographic Information Systems (GIS) is to bridge the gap between detailed building design and the broader geospatial context (Azari et al., 2025; Noardo et al., 2020). While BIM and GIS were developed for different purposes, their integration supports the transformation of information into actionable knowledge, which is essential for Smart City and Digital Twin applications (Azari et al., 2025). the integration addresses specific limitations: BIM typically lacks information about the surrounding environment (e.g., terrain and neighboring buildings), while GIS usually lacks detailed internal and semantic building data. Combining both enables more comprehensive management of the built environment across the entire life cycle, from planning and design to operation and maintenance.

BIM provides a digital representation of a facility's physical and functional characteristics with a high level of detail (LoD). It focuses on the design, construction, and asset management of individual buildings or infrastructure, typically using a local Cartesian coordinate system (Noardo et al., 2020). BIM also contains rich semantic data about building components (e.g., materials, costs, and schedules).

GIS is used to manage and analyze spatial data at larger scales, such as cities, landscapes, or regions. It enables location-based analysis for tasks such as environmental assessment, topographic analysis, and urban planning. GIS commonly relies on global coordinate reference systems to locate objects accurately on the Earth's surface (Azari et al., 2025).

A key technical challenge in BIM–GIS integration is therefore the georeferencing step, i.e., aligning BIM's local coordinates with GIS's global reference frame in a consistent and reproducible way (Azari et al., 2025; Noardo et al., 2020). When this alignment is handled correctly, integration can support applications such as city model updates, improved site selection, and combined indoor–outdoor analyses that strengthen infrastructure planning and facility operations(Azari et al., 2025).

2.4 Internet of Things (IoT) and sensor systems in buildings

The Internet of Things (IoT) connects the physical world with the digital world through sensing and actuating devices that exchange information across platforms to support a shared operational view (Herlé & Blankenbach, 2024). In this context, a physical object can be treated as an asset that carries information throughout its life cycle, supporting a shift from product-oriented to service-oriented processes (Abdelalim et al., 2024).

IoT functionality can be categorized into two main types:

- Sensing: sensors observe or measure physical phenomena, for example humidity, temperature, pressure, CO_2, occupancy, structural health monitoring, or CCTV (Abdelalim et al., 2024).
- Tasking (actuation): mechanisms that allow systems to command an actuator or device to perform an action, such as changing the color of an LED lamp or controlling equipment (*OGC SensorThings API Documentation | SensorUp OGC SensorThings API Developer Centre*, n.d.).

To manage the complexity of connecting diverse devices, architectures and standards are commonly used. One example is the OGC SensorThings API (Open Geospatial Consortium), an open standard that models the physical world using entities such as Things (devices), Locations, Datastreams (collections of observations), Sensors, and Observations (the measured readings). It supports standard CRUD operations (Create, Read, Update, Delete) through a RESTful interface over HTTP/HTTPS (*OGC SensorThings API Documentation | SensorUp OGC SensorThings API Developer Centre*, n.d.).

The integration of BIM (static information) with IoT (dynamic time-dependent data) enables Digital Twin applications by providing a near real-time virtual representation of a physical asset. A typical workflow includes the following steps (Abdelalim et al., 2024)

- Mapping: IoT sensors are physically installed and digitally linked to BIM elements using unique identifiers (GlobalIDs).

- Visualization: Real-time sensor data (e.g., a room getting too hot) is displayed directly on the 3D BIM model, allowing operators to locate and address issues more efficiently.
- Predictive Maintenance: Instead of reacting to failures, the system can use data analytics (Machine Learning) to estimate when equipment (e.g., HVAC components) may require maintenance based on actual usage rather than fixed schedules.
- Benefits: This integration can lead to cost savings, improved energy efficiency, enhanced occupant comfort, and more streamlined FM operations by reducing manual inspection efforts and improving information availability (Abdelalim et al., 2024).

2.5 Web – based 3D visualization and Digital Twin concepts

The field has evolved from standalone desktop applications to web-based solutions utilizing WebGL standards. This shift allows for the rendering of complex 3D graphics directly in web browsers without the need for plugins (Stojanovic, 2021).

The advantage is that this approach decouples hardware requirements from the client device (enabling access via "thin clients" like mobile devices), supports platform independence, facilitates multi-user collaboration, and allows for easier integration with centralized servers for real-time data streaming (Auer et al., 2014).

Key Libraries and Ecosystems

Web-based 3D visualization is commonly implemented using JavaScript libraries that build on WebGL. Three.js is a high-level, open-source library that abstracts low-level graphics programming and provides a scene graph, materials, lighting, camera controls, and interaction utilities, enabling interactive 3D applications directly in the browser (Stojanovic, 2021). In the context of BIM visualization, such libraries are frequently used to render detailed building geometry and support user interactions such as element selection, inspection, and measurement.

For geospatial context and large-scale visualization, CesiumJS is widely used as an open-source platform for 3D globes and maps, supporting terrain, imagery layers, and georeferenced 3D content. Combining a general-purpose 3D engine (e.g., Three.js) with a geospatial engine (e.g., CesiumJS) enables workflows in which detailed BIM content can be explored together with its surrounding environment and location-dependent information, which is particularly relevant for Digital Twin applications (Stojanovic, 2021).

3. Related Work and Existing Frameworks

This chapter reviews relevant work and existing frameworks at the intersection of Facility Management (FM), BIM/IFC, GIS, IoT, and web-based visualization. First, it summarizes how established CAFM/CMMS and IWMS platforms currently support FM workflows and how BIM is typically integrated in practice. It then discusses research efforts that use IFC and open standards to improve interoperability and enable bidirectional updates. Next, it covers BIM–GIS integration approaches at campus/city scale, followed by BIM–IoT Digital Twin frameworks. Finally, the chapter outlines the recent open-source ecosystem for IFC on the web and derives the key gaps and requirements that motivate the proposed framework in this thesis.

3.1 BIM – based FM tools and CAFM/CMMS systems

In practice, Facility Management (FM) relies heavily on established digital tools. CAFM (Computer-Aided Facility Management) systems are mainly used for space management, administration, and room inventories (Parsanezhad & Dimyadi, 2014). CMMS (Computerized Maintenance Management Systems) focus on asset management, service requests, and maintenance resources. More comprehensive commercial platforms often described as Integrated Workplace Management Systems (IWMS) attempt to combine these functions (e.g., IBM Maximo, ARCHIBUS, FM:Interact).(Wijeratne et al., 2024)

Some typical functionalities of these systems provide mature administrative capabilities, including:

- Asset and equipment management: Tracking inventory, installation dates, and warranty details (Parsanezhad & Dimyadi, 2014).
- Work orders: Managing the lifecycle of service requests and corrective maintenance tasks (Wijeratne et al., 2024).
- Preventive maintenance planning: Scheduling routine inspections and replacements to avoid failures.(Wijeratne et al., 2024).
- Space and cost management: Administering room areas, departments, contracts, and operational costs (Wijeratne et al., 2024).

The integration of BIM into these existing systems varies in depth and method. Many systems allow users to attach 2D plans or 3D BIM models as hyperlinks or through "portal" solutions that act as windows to the model. Regarding data exchange, some platforms export or read COBie (Construction Operations Building information exchange) spreadsheets to populate asset registers(Wijeratne et al., 2024). Others import IFC files, though this capability is often limited or results in large files containing irrelevant information for operations. In practice, BIM–FM integration is usually partial; the BIM model is not the core database but rather serves as a visual reference or a repository for populating the CMMS during handover(Parsanezhad & Dimyadi, 2014).

Some typical problems that literature identifies is the lack of interoperability between BIM author tools and FM systems (CAFM/CMMS), leading to data loss during transfer. Another problem is that the industry struggles with a wide variety of proprietary tools varying data structures that hinder seamless exchange.(Parsanezhad & Dimyadi, 2014; Volk et al., 2014) Gathering all of that, there are some inefficient workflows because uploading data often involves manual work, such as overlaying polygons on drawings or manually querying

database, making the process of keeping BIM models up-to-date with operational changes difficult and time consuming.(Marmo et al., 2020)

These systems show that FM software is mature on the administrative side, but BIM is rarely used as the primary information backbone. Instead, BIM is mostly treated as an optional viewer or one-way data source.

3.2 Research on BIM – FM integration using IFC

Scholarship regarding BIM–FM integration has decisively shifted focus from proprietary, vendor-specific middleware towards open-standard frameworks designed to ensure long-term data accessibility. This transition is primarily driven by the need to mitigate the risks of vendor lock-in and to foster interoperability across heterogeneous software environments (Otranto et al., 2025). Within this OpenBIM paradigm, the Industry Foundation Classes (IFC) schema is frequently adopted as the central repository for asset information, serving as a non-proprietary bridge between design and operations.

Recent technical implementations have sought to overcome the limitations of raw IFC files. For instance, (Matarneh et al., 2022) proposed a novel cloud-based architecture that parses IFC files directly into relational SQL databases; this approach automates the population of asset registries and significantly reduces the data redundancy and human error inherent in traditional manual COBie workflows. Addressing a different challenge—specifically the computational weight of monolithic models (Herlé & Blankenbach, 2024) introduced a hypermedia-driven RESTful API. Their framework decomposes complex IFC entities into granular web resources, facilitating dynamic, lightweight interactions between BIM, GIS, and IoT data without the latency associated with transferring entire model files.

Despite these advancements, the transition from theoretical prototypes to practical application faces significant hurdles. As highlighted by (Wijeratne et al., 2024) in a comprehensive review of 33 implementation cases, the semantic mapping between Facility Management ontologies and rigid IFC structures is rarely seamless, often resulting in critical data loss during export. Moreover, the issue of 'round-tripping'—synchronizing operational updates back into the original design model—remains a persistent technical challenge that threatens the data integrity of the digital twin over its lifecycle (Noardo et al., 2020) .

3.3 BIM – GIS and city – scale integration approaches

The integration of BIM and GIS has evolved from theoretical data mapping to the development of practical frameworks capable of managing built assets at the campus and city scales. A fundamental challenge in this domain remains the harmonization of spatial data; as (Azari et al., 2025) emphasize in their recent review, aligning the local Cartesian coordinates of BIM with the global geodetic systems of GIS requires structured transformation methods to prevent positioning errors that compromise utility mapping. Furthermore, to handle the massive geometric complexity of BIM models in web-based GIS environments, researchers like (Wang et al., 2024) have proposed advanced algorithms for building envelope extraction, effectively reducing data weight while maintaining the necessary Level of Detail (LOD) for urban visualization.

Moving beyond geometry to semantic interoperability, recent scholarship highlights the efficacy of ETL (Extract, Transform, Load) processes for campus management. A notable

implementation is the work of (Savalskii, 2025), who developed a comprehensive workflow for the HFT Stuttgart campus. By utilizing FME to map IFC data to the CityGML standard and deploying a 3D City Database (3DCityDB) linked to a CesiumJS frontend, this study demonstrated how BIM data can be successfully visualized and queried within a web-based Digital Twin environment. Such frameworks are increasingly extended to address broader urban challenges; for instance, (Barrile et al., 2024) illustrate how integrating 4D BIM data into GIS platforms can drive sustainability assessments and resilient city planning. However, a recurring limitation in these city-scale approaches is that while they excel in urban planning, they often sacrifice the granular, component-level data required for daily Facility Management operations.

3.4 BIM – IoT / Building Digital Twin Frameworks

The convergence of Building Information Modeling (BIM) with the Internet of Things (IoT) has fundamentally reshaped the Facility Management landscape, driving a paradigm shift from static 3D representations toward dynamic, data-driven Digital Twins. This transition is substantiated by comprehensive bibliometric analyses conducted by Shi(2025) and Hakimi et al. (2024), which indicate an exponential surge in research output over the last five years. Their findings reveal that the industry focus has rapidly matured from simple data visualization to complex Cyber-Physical Systems (CPS) capable of predictive maintenance and real-time anomaly detection.

In response to these demands, recent scholarship has produced increasingly sophisticated integration frameworks. For example, (Nguyen et al., 2025) developed a multi-layered Digital Twin architecture specifically designed for relocatable modular buildings. By orchestrating data across BIM, GIS, and IoT domains, their framework addresses the unique lifecycle challenges of temporary infrastructure managing not just daily operations but also the logistics of disassembly and relocation.

However, a parallel stream of research argues against the necessity of such capital-intensive platforms, advocating instead for the democratization of Digital Twins through open standards. (Chatsuwan et al., 2025) successfully demonstrated this by constructing a lightweight, low-cost framework for Indoor Environmental Quality (IEQ) monitoring. Utilizing open-source tools like Node-RED and JSON to stream sensor data into an IFC-based viewer, they proved that HVAC optimization is achievable without relying on proprietary vendor ecosystems.

Despite these advancements, a critical technical bottleneck remains unresolved: the structural limitation of the IFC schema itself. As argued by (Wills & Bartels, 2022) , while IFC serves as an excellent standard for static semantic data, it is not natively architected to handle high-frequency time-series data generated by IoT sensors. Consequently, most OpenBIM Digital Twins still require complex middleware to bridge the gap between the static model and dynamic operational data, preventing a truly seamless data environment.

3.5 Open – Source ecosystem for IFC on the web

Over the last decade, the open-source ecosystem surrounding the Industry Foundation Classes (IFC) standard ("Industry Foundation Classes (IFC)," n.d.) has matured from experimental research into a robust infrastructure capable of supporting enterprise-grade web applications. On the server side, libraries such as IfcOpenShell (IfcOpenShell, 2015/2026) have become the de facto standard for programmatic access to BIM data, allowing developers

to traverse complex spatial hierarchies and extract property sets without reliance on proprietary 'black-box' middleware. Simultaneously, the frontend landscape has been revolutionized by the adoption of WebAssembly (Wasm). Projects like *web-ifc* and its integration with *Three.js* spearheaded by initiatives like *That Open Company* (That Open, 2021/2026; That Open, 2020/2026) now enable browsers to parse and render gigabyte-scale models with near-native performance, supporting advanced interaction patterns like element picking and boolean clipping.

However, despite the availability of these powerful modular components, a significant gap remains in their orchestration for Facility Management (FM). Current implementations often function as isolated viewers rather than comprehensive management platforms; they rarely offer a unified API architecture to handle the heterogeneous nature of building data. To bridge this divide, modern software engineering standards—such as OpenAPI (*OpenAPI Initiative – The OpenAPI Initiative Provides an Open Source, Technical Community, within Which Industry Participants May Easily Contribute to Building a Vendor-Neutral, Portable and an Open Specification for Providing Technical Metadata for REST APIs – the "OpenAPI Specification" (OAS).*, n.d.) for interface description, GeoJSON (RFC 7946) (Butler et al., 2016) for geospatial context, and high-performance frameworks like FastAPI (*FastAPI*, n.d.)—must be integrated to transform raw IFC geometry into a living asset database. Consequently, the challenge is no longer the visualization of the model, but the development of a coherent framework that binds these open-source tools into a system capable of managing work orders, sensor streams, and user permissions efficiently.

3.6 Summary of gaps and requirements for the thesis

This review shows that commercial CAFM/CMMS and IWMS solutions provide mature administrative functionality, yet BIM is often treated as an optional viewer or a one-way handover source rather than the primary operational information backbone. Research prototypes increasingly adopt IFC and open standards to improve interoperability and reduce vendor lock-in, but practical deployment is still constrained by imperfect semantic mapping, limited round-tripping back into IFC, and the computational weight of full-model transfers. In parallel, BIM–GIS approaches enable campus/city-scale context, but they frequently sacrifice component-level detail that is essential for daily FM tasks. BIM–IoT Digital Twin frameworks demonstrate the value of real-time monitoring; however, IFC remains fundamentally optimized for static semantic data and typically requires middleware to connect high-frequency sensor time-series to building semantics.

From an implementation perspective, the open-source ecosystem for IFC on the web has matured significantly (IfcOpenShell server-side and WebAssembly-based IFC parsing client-side). Nevertheless, existing solutions are commonly delivered as isolated viewers without a unified architecture for operational workflows such as work orders, notes, and sensor monitoring. Therefore, the main gap is no longer "whether IFC can be visualized in the browser", but how to orchestrate BIM, GIS, and IoT through a coherent, standards-oriented backend that supports discoverable resources and lifecycle updates.

Based on these gaps, this thesis derives the following requirements for the proposed framework:

- R1 — IFC as semantic backbone: use stable IFC GlobalId (GUID) identifiers and IFC relationships to anchor FM entities and maintain consistent linking across sessions.

- R2 — Web-native, API-centered architecture: expose BIM/FM functionality through a structured REST API (hypermedia/HATEOAS-style where applicable) to ensure operations are explicit, discoverable, and traceable.

- R3 — BIM–GIS integration for context: support geospatial placement and a calibration workflow for non-georeferenced models to enable campus-scale understanding.

- R4 — IoT integration beyond visualization: provide a clean ingestion/streaming path for sensor updates and map observations to IFC spaces/elements without forcing sensors to “speak IFC”.

- R5 — Prototype usability for FM workflows: implement CRUD for notes/work orders/assets linked to IFC semantics to demonstrate day-to-day FM interactions, not only model viewing.

4. Methodology and Research Design

This chapter describes the methodological approach used to design, implement, and evaluate the proposed open-source framework. It explains the research approach, the scope and constraints, how requirements were derived and traced to implementation, the development strategy, the case study setup, and the evaluation procedure used to validate the artifact.

4.1 Research approach

To achieve the objectives of this thesis, a Design Science Research (DSR) methodology was adopted. This approach is particularly suitable here because the primary contribution is a practical software artifact: an open-source framework integrating BIM (IFC), GIS, and IoT. Rather than relying solely on theoretical analysis, the research focuses on the iterative development of a functional prototype. Through cycles of building, evaluating, and refining, the system is designed to address real-world Facility Management needs, such as handling work orders and monitoring sensor data.

4.2 Problem framing and scope

Within the scope of this thesis, I present a proof-of-concept framework that bridges the gap between IFC-based models and operational web visualization. The intention is not to compete with fully matured CMMS (Computerized Maintenance Management Systems) solutions, but to provide a modular open-source alternative that researchers and developers can extend. Consequently, the implementation concentrates on three core pillars: semantic linking using the GlobalId, placing buildings in context with a 3D globe, and supporting essential FM interactions. By clearly stating these limitations, the research avoids overclaiming functionality while validating the technical architecture.

4.3 Requirements engineering

To ensure the prototype remains both practically useful and technically feasible, I structured the system criteria into functional vs non-functional requirements. The functional aspect focuses on user capabilities such as IFC visualization, element selection, and sensor monitoring. Conversely, the non-functional side strictly adheres to open-source principles, ensuring interoperability and maintaining high performance even with large files. To guarantee that these goals were met, I established a traceability strategy that maps each abstract requirement to a specific code module, bridging the gap between the thesis objectives and the final implementation.

4.4 Development Methodology

The development of the framework followed a rigorous iterative prototyping strategy, structured around clear incremental milestones. This approach allowed for the gradual introduction of complex features, such as IoT integration, ensuring that each new addition was stable before moving forward. To maintain code quality, I relied on Git (https://github.com/Jodavecu/tesis-ifc-viewer.git) for version control, which facilitated the tracking of changes and the application of necessary regression fixes whenever updates caused conflicts. Furthermore, the system architecture was designed to be strictly modular, a critical decision that keeps the handling of BIM, GIS, and sensor data distinct and extensible.

4.5 Case study and dataset

The framework was validated through a real-world case study situated within an HFT campus deployment. The dataset comprises IFC models providing detailed building geometry and semantics, combined with geospatial context layers for precise campus-scale placement. For operational monitoring, the prototype integrates real sensor time-series (covering metrics such as CO_2, temperature, and humidity) obtained from an existing installation. However, regarding dataset provenance, it is important to note that the sensor data originated from lecture rooms in a different building than the IFC model used for the main demonstration. To address this practical constraint, the evaluation explicitly distinguishes between validating the replay mode of the ingestion pipeline and the semantic interactions on the available IFC model. This approach reflects common industry challenges where sensor inventories and BIM deliverables are not co-located or synchronized.

4.6 System integration workflow (end to end)

The prototype was implemented as an end-to-end workflow that connects BIM, GIS, and IoT without relying on proprietary formats. On the client side, WebAssembly (WASM) is used to parse IFC models locally for responsive interaction. Operational records are anchored to the IFC GlobalId backbone for consistent linking. When BIM and sensor inventories are not co-located, the system supports a replay-based integration mode that validates the ingestion and streaming workflow independently from full BIM-to-sensor spatial mapping. Sensor integration uses Server-sent events (SSE) streaming to push updates directly to the browser for visualization. The workflow also accounts for real-world data imperfections by including a manual calibration step for non-georeferenced models. Finally, the data layer follows a hybrid persistence strategy, combining a relational database with filesystem artifacts to support rapid prototyping and reproducible deployments.

4.7 Evaluation Strategy

Evaluation was conducted through a set of reproducible test scenarios designed to approximate typical FM usage. Scenarios included loading large IFC models, retrieving element properties, using inspection tools, and monitoring near real-time sensor streams. This served both as a feature demonstration and a technical validation of the architecture. Qualitative and quantitative indicators were recorded, including perceived responsiveness during navigation and the streaming latency of IoT updates. Each scenario was mapped back to the functional requirements to maintain traceability between objectives and evidence.

Evaluation was performed in two layers: (1) functional validation of BIM-linked FM tasks on the available IFC model, and (2) validation of the IoT pipeline using real sensor datasets in replay mode, given the lack of a matching IFC for the sensor-equipped building.

4.8 Data Governance, Ethics, and Constraints

Security and ownership were prioritized by designing for data sovereignty, allowing the entire stack to run on-premise. This ensures that critical facility data never leaves the owner's infrastructure. On the operational side, I treated data quality realistically, acknowledging that sensor calibration drift is a common challenge in FM. These technical limitations are clearly documented to contextualize the results. A key constraint was the partial mismatch between available BIM and sensor datasets: while the sensor data is real, it originated from a different

campus building than the IFC model used for the main prototype demonstration. This is documented as a practical limitation and motivates future work on full BIM–sensor co-registration. Additionally, the prototype focuses solely on ambient conditions to avoid the privacy concerns typically associated with occupancy tracking.

4.9 Summary of methodology

To summarize, this work adopts a DSR framework to bridge the gap between theoretical design and practical application. The process was driven by iterative development, using requirements traceability as a control mechanism to map abstract goals to concrete code. Validation was conducted within a real-world case study environment, utilizing scenario-based evaluation to assess the system's performance in typical facility management scenarios. This methodological rigor provides the necessary evidence to confirm that the artifact effectively solves the identified research problems.

5 Framework Design and System Architecture

This chapter presents the design rationale and system architecture of the proposed open-source framework for web-based Facility Management (FM). The framework is intentionally modular and standards-oriented: IFC acts as the semantic backbone for building information, a geospatial engine provides context at campus scale, and operational data (FM records and sensor readings) is integrated through lightweight web APIs and near real-time streaming. The chapter is structured around (i) framework requirements, (ii) a conceptual data model, (iii) overall architecture across BIM/GIS/IoT layers, (iv) integration strategies, and (v) security and open-source considerations.

The overall system architecture of the proposed framework is summarized in Figure 5-1, highlighting the main modules and data flows between the web client, backend services, and data sources.

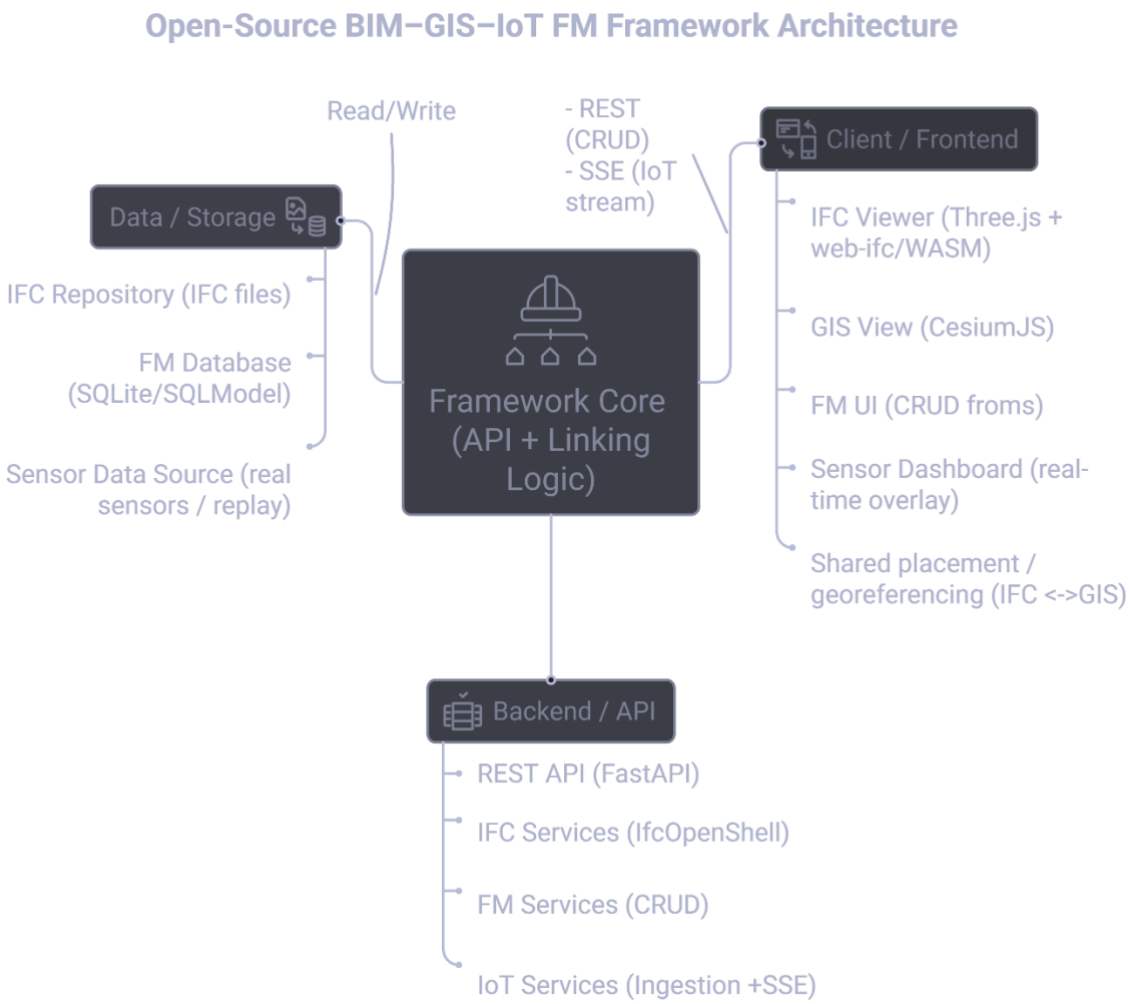

Figure 1 Architecture

Figure 5-1: High-level architecture of the proposed open-source BIM–GIS–IoT Facility Management framework. The client consumes REST endpoints for CRUD operations and receives sensor updates via Server-Sent Events (SSE). IFC models are accessed from an IFC repository, while FM entities are persisted in a database.

5.1 Requirements for an open-source FM framework

The framework was designed to address common limitations of proprietary BIM–FM solutions, including recurring licensing costs, restricted interoperability, vendor lock-in, and limited support for integrating operational sensor data. Requirements are grouped into functional and non-functional requirements to reflect both the intended FM workflows and the engineering constraints of web-based BIM systems.

Functional requirements (FR)

- **FR1 – Web-based IFC visualization and interaction.** The system shall load and render IFC models in a standard web browser and support interaction patterns relevant to FM, including orbit/navigation, element selection, and basic inspection of element properties.
- **FR2 – FM CRUD linked to IFC semantics.** The system shall provide create/read/update/delete (CRUD) operations for FM entities (e.g., notes and work orders) and associate each record with a specific IFC element via its persistent GlobalId.
- **FR3 – Semantic filtering and view modes.** The system shall support visibility control and thematic views (e.g., architectural vs. MEP/technical view) based on IFC semantics and system relationships, enabling users to focus on specific categories or building systems.
- **FR4 – Geospatial context and site awareness.** The system shall embed BIM visualization within a geospatial context (campus/terrain), enabling the building model to be understood relative to its surroundings rather than as an isolated object.
- **FR5 – Sensor integration and near real-time updates.** The system shall integrate sensor data relevant for building operation (e.g., CO_2, temperature, humidity) and provide a near real-time update mechanism in the web client to support monitoring and FM decision-making.
- **FR6 — Clash detection.** The system shall support basic clash detection to identify geometric intersections between selected building elements (e.g., between MEP and structural components) and report detected collisions to the user for inspection.
- **FR7 — Measurements (distance/area).** The system shall provide interactive measurement tools in the web client, enabling users to measure distances and areas directly on the 3D model to support inspection and on-site verification tasks.
- **FR8 — Model comparison (diff)**. The system shall support a lightweight model comparison workflow, allowing users to compare two model states (or versions) and highlight differences (e.g., added/removed/changed elements) to assist in change tracking.
- **FR9 — Clipping and sections**. The system shall provide clipping/sectioning functionality, allowing users to define section planes or volumes to inspect internal components and spatial relationships within the building model.
- **FR10 — File upload.** The system shall allow users to upload IFC models to the platform, store them on the server, and make them available for subsequent loading and visualization in the web client.

Non-functional requirements (NFR)

- **NFR1 – Open-source-first and cost-effective.** The framework shall rely exclusively on open-source components to eliminate licensing fees and ensure transparency and auditability.
- **NFR2 – Interoperability through open standards.** The framework shall use IFC (OpenBIM) as the authoritative building data source and expose operational information through standard web technologies (HTTP/REST). For IoT, the framework shall remain compatible with established patterns (e.g., publish/subscribe messaging and SensorThings-like entity models), even if the prototype implements a simplified subset.
- **NFR3 – On-premise deployability and data sovereignty.** The system shall support local/on-premise deployment so that building models and operational data remain under the control of the asset owner.
- **NFR4 – Performance and responsiveness for large models.** The system shall remain usable for large IFC models (e.g., >100MB) by leveraging browser-side acceleration (WebAssembly) and avoiding UI blocking operations.
- **NFR5 – Modular separation of concerns.** The architecture shall clearly separate responsibilities: the backend provides semantics, persistence, and heavy processing; the frontend provides interaction and visualization.

Req ID	Requirement (Summary)	Frontend Module (Actual)	Backend Module (Actual)	Code Evidence
FR1	IFC Visualization	ifcLoader (in main.js)	IfcRouter (routers/ifc.py)	WASM loading & geometry parsing via web-ifc
FR2	IFC Visualization	CesiumMap (components/CesiumMap.js)	GisRouter (routers/gis.py)	ArcGIS/OSM integration + Coordinate sync
FR3	IoT Sensor Management	IoTVisualization (iot-visualization.js)	IoTService (services/iot_service.py)	3D spheres & heatmap visualization
FR4	BIM Editing (Move/Rotate)	BimEditor (components/BimEditor.js)	IfcModifierService (services/ifc_modifier.py)	ransform Gizmo + Python apply_deltas logic
FR5	Element Tree	ModelTree (model-tree.js)	IfcService (services/ifc_service.py)	Spatial structure (Site → Storey → Element)

FR6	Clash Detection	ClashManager (components/Clash Manager.js)	N/A (Client-side processing)	computeIntersection using Three.js BVH
FR7	Measurements (Dist/Area)	MeasurementsManager (measurements.js)	N/A (Client-side processing)	Raycaster implementation + CSS2D Labels
FR8	Model Comparison (Diff)	ModelDiffManager (components/Model DiffManager.js)	N/A (Client-side processing)	Visual comparison of subsets by GlobalId
FR9	Clipping & Sections	ClippingManager (clipping-manager.js)	N/A (Client-side processing)	WebGL clipping planes implementation
FR10	File Upload	UIManager (File Input handler)	SitesRouter (routers/sites.py)	/sites/upload endpoint & file storage

Table 1 Functional

The figure above: illustrates the overall architecture of the proposed open-source Facility Management framework. The client implements a hybrid visualization approach: CesiumJS provides a geospatial reference frame (globe/terrain and global camera context), while Three.js renders high-detail BIM geometry and interactive overlays in a synchronized layer. IFC data is parsed in the browser using a WebAssembly-based engine to enable responsive selection and inspection. Operational functionality (FM CRUD, project handling, and heavy IFC mutations) is provided by a FastAPI backend, which also acts as an IoT mediator: it ingests sensor readings and pushes near real-time updates to the client via Server-Sent Events (SSE). Persistence follows a prototype-oriented hybrid strategy combining SQLite/SQLModel for structured FM metadata and filesystem/JSON storage for project files, IFC models, and sensor snapshots.

In later chapters, these requirements are mapped to implemented modules and evaluation evidence (e.g., feature demonstrations and user-task walkthroughs) to ensure traceability between design goals and the delivered prototype.

5.2 Conceptual data model

The conceptual model is deliberately minimal to keep the prototype lightweight and extensible. It focuses on three core domains: (i) IFC elements and spatial structure (GlobalId, IfcSite/Building/Storey/Space), (ii) FM entities (e.g., assets, notes, work orders) stored in the database, and (iii) IoT observations (sensor readings with timestamps and units). The GlobalId (GUID) acts as the primary linking key that associates FM records and sensor streams to specific BIM components, while additional metadata (e.g., location, sensor type, thresholds) remains optional and can be extended in future work.

Advanced FM taxonomies (e.g., full COBie structures, multi-site tenancy, or workflow engines) are out of scope for this thesis and are considered future extensions.

5.2.1 IFC as semantic backbone (GlobalId, spaces, systems)

The Industry Foundation Classes (IFC) schema is used not only to represent geometry, but to provide the semantic structure upon which FM entities and sensor data can be anchored.

GlobalId as the backbone key. IFC GlobalId is treated as the stable external identifier for building elements. FM records (e.g., notes, work orders) are associated with the corresponding element GlobalId, allowing operational data to remain consistently linked even across model reloads or geometry regeneration.

Spatial hierarchy and containers. The framework reconstructs the standard IFC spatial hierarchy (IfcProject → IfcSite → IfcBuilding → IfcBuildingStorey → IfcSpace) to enable navigation and context-aware features. IfcSpace provides a natural container for room-level aggregation and is a key anchor for operational data that is more meaningful at room scale (e.g., indoor environmental quality sensors).

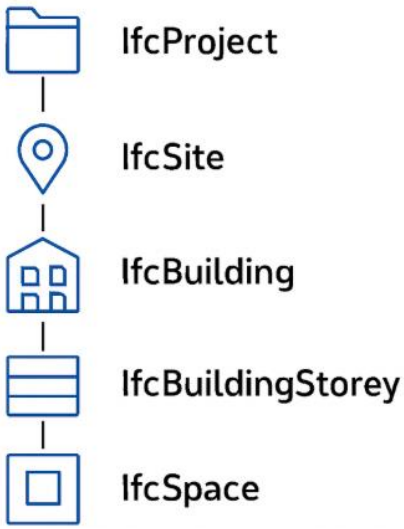

Figure 2 Spatial hierarchy

System semantics for technical views. Building services and Mechanical, Electrical, and Plumbing engineering (MEP) components are organized using IFC system relationships (e.g., distribution systems and related MEP classes). These relationships support view modes that separate architectural layers from technical systems, improving visual inspection and troubleshooting during FM task such as troubleshooting and maintenance planning.

5.2.2 FM entities: notes, work orders, assets, sensors

To support FM workflows beyond passive model inspection, the framework introduces operational entities linked to IFC.

FM entities (CRUD-capable). The core FM entities include:

- Notes: text annotations linked to a target IFC element via global_id, typically including creation time and author metadata if available.
- Work orders: structured tasks linked to an IFC element via global_id, including title, description, priority and status, and timestamps.

These entities represent the minimum viable set needed to demonstrate FM-style interactions in a BIM context (recording observations, creating tasks, tracking completion).

Sensors and observations (time-varying data). Sensor integration is represented through a simplified, Sensor Things-inspired structure:

- Thing: contextual “asset” or location the sensor belongs to (e.g., a room or device context),
- Sensor: hardware identity and metadata,
- Data stream: logical container describing the measured phenomenon and unit,
- Observation: individual time-stamped measurements.

The prototype implements this model in a lightweight, prototype-oriented way, focusing on the entity fields required for mapping and visualization. A complete, standards-compliant SensorThings service is treated as a future extension rather than a prerequisite for the current prototype stage.

5.2.3 Hybrid persistence architecture

The prototype adopts a hybrid persistence architecture. A relational database (SQLite via SQLModel) is used for structured, low-volume data such as configuration metadata and core FM records. In parallel, the filesystem is used to store file-centric and high-volume artifacts:

- raw IFC models (.ifc) and project assets,
- IoT datasets and sensor inventories stored as JSON snapshots (e.g., sensors_*.json).

This design reduces complexity during rapid prototyping and provides transparent debugging of IoT ingestion and streaming. It also preserves IFC files as authoritative artifacts, while keeping a clear migration path towards a fully database-backed persistence layer (e.g., PostgreSQL/PostGIS) when the schema stabilizes.

5.3 Overall system architecture (BIM + GIS + IoT)

The system is structured as a three-tier architecture: presentation layer (client), application layer (API), and data layer (storage). This separation is central to modularity and reusability across different buildings and deployment contexts.

5.3.1 Presentation layer (client)

The final client implements hybrid visualization:

- CesiumJS provides the global geospatial reference frame, terrain, and camera context.
- Three.js renders detailed BIM geometry and interactive overlays in a synchronized layer above the geospatial engine.

IFC parsing and geometry generation are performed in the browser using a WebAssembly-based IFC engine, enabling interactive selection and inspection without continuous server

round-trips. Leaflet is retained only as a secondary minimap/logistics view in a small floating container, rather than as the primary visualization environment.

5.3.2 Application Layer (API)

The backend exposes a web API with endpoints to support:

- access to projects and IFC models,
- semantic queries (properties, spatial hierarchy, element metadata),
- CRUD operations for FM entities,
- IoT mediation (ingestion and streaming),
- backend responsibilities that are better handled server-side, such as persistence, validation, access control, and long-running semantic queries on large IFC models.

In practice, the browser focuses on interactive rendering and lightweight queries, while the backend handles validation, persistence, and compute- or security-sensitive processing.

5.3.3. Data Layer (storage)

The prototype uses a hybrid persistence strategy to keep the system lightweight and easy to deploy and maintain, while supporting both structured FM records and large BIM-related artifacts. Relational storage is used for frequently queried entities and configuration data, whereas file-based storage is used for bulky and versionable artifacts such as IFC models and JSON snapshots.

Persistence combines:

- filesystem storage for raw models and JSON-based prototype datasets under a dedicated data/ directory.
- SQLite/SQLModel for structured FM entities and configuration metadata.

This hybrid approach reflects the current prototype stage and supports both reproducible deployments (models and JSON snapshots can be versioned and archived) and efficient FM lookups through relational indexing.

5.4 Integration strategies

A central design goal is to integrate BIM, GIS, and IoT without conflating responsibilities or relying on fragile, tool-specific assumptions. Integration is therefore handled through explicit mapping strategies.

5.4.1 IFC ↔ GIS

The BIM model is aligned to the geospatial environment when IFC georeferencing information is available. When georeferencing is missing or incomplete, the system provides a manual calibration workflow to anchor the model to the map by applying an offset/transform at the root level. In the hybrid renderer, Cesium controls the global camera pose and reference frame, while Three.js renders BIM geometry in a local coordinate system that is continuously

synchronized to the geospatial camera. This approach preserves the high-precision local geometry of BIM while benefiting from global context and terrain rendering.

5.4.2 IFC ↔ sensors

Sensor integration follows a decoupled mapping strategy:

1. Ingestion: sensors provide readings via lightweight protocols (e.g., Message Queuing Telemetry Transport (MQTT) or HTTP-based ingestion), independent of BIM.
2. Mapping: the backend maintains a mapping between sensor identifiers and BIM semantics (e.g., IfcSpace or element GlobalId), enabling room-level or element-level association.
3. Streaming to client: updates are broadcast to the web client via Server-Sent Events (SSE) to enable near real-time visualization.
4. Visualization: the client updates markers, overlays, and/or semantic color encodings in the BIM/GIS view based on incoming observations.

This strategy supports both real sensor deployments and simulated sensor data without requiring the sensor devices to “know” anything about IFC.

5.5 Security, data ownership and open-source considerations

Building models and operational sensor data can be sensitive, as they may reveal spatial layouts, usage patterns, or operational weaknesses. Therefore, the framework prioritizes on-premise deployment and data sovereignty: IFC files, FM records, and sensor datasets can remain under institutional control rather than being uploaded to vendor-managed cloud environments. The exclusive reliance on open-source components further improves transparency and auditability, since the implementation and data flows can be inspected and verified. Finally, by relying on open standards (IFC and web-based geospatial/IoT formats and protocols), the framework supports long-term accessibility and reduces vendor lock-in, avoiding dependency on proprietary file formats that may become obsolete or restricted over time.

6 Implementation

This chapter describes how the conceptual architecture presented in Chapter 5 was implemented as a functional prototype. The implementation follows an MVP (Minimal Value Product) oriented full-stack approach, prioritizing reproducibility, modularity, and demonstrable end-to-end functionality: a lightweight backend exposes BIM/GIS/IoT services via HTTP and streaming endpoints, while the web client renders the 3D scene, supports interaction tools, and synchronizes operational data in near real time. In this context, *MVP-oriented* means implementing only the components required to validate BIM–GIS–IoT linking, CRUD operations, and near real-time sensor updates. Throughout development, priority was given to keeping the system lightweight, transparent, and easy to deploy, while preserving the ability to evolve the prototype toward a maintainable open-source framework.

6.1 Backend implementation (FastAPI + IfcOpenShell)

The backend was implemented as an asynchronous web service using FastAPI (Python 3.10+). FastAPI was selected due to its native support for async/await, strong request validation via Pydantic/SQLModel, and its suitability for I/O-bound workloads (file access, database queries, network calls) while keeping the application responsive during computational geometry operations. The backend acts as the coordination layer of the framework: it manages projects and uploads, exposes BIM metadata services, provides GIS context data, and mediates IoT streaming to the frontend.

6.1.1 Project structure and main components

The repository follows a clean separation of concerns aligned with a Router–Service–Model structure:

- app/core: central configuration and shared instances (CORS policy, database session, settings).
- app/routers: HTTP interface definition (REST endpoints), including:
  - ifc.py: endpoints for model metadata, spatial hierarchy, and element queries.
  - sites.py (or equivalent): project handling and file upload/import logic.
  - iot.py: endpoints for sensor retrieval and live streaming.
  - gis.py: proxy services to retrieve contextual GIS data (e.g., building footprints).
- app/services: backend business logic encapsulation. For example, ifc_modifier.py contains controlled routines that apply transformations and persist changes safely.
- app/models: data schemas used both for API validation and persistence (via SQLModel, combining Pydantic + SQLAlchemy patterns).

This structure ensures that endpoint code remains thin (routing, validation), while the domain logic lives in testable service functions. As a result, changes in one part of the system (e.g., sensor ingestion formats) can be handled without coupling to unrelated modules.

6.1.2 IFC services (loading, property queries, spatial structure)

The BIM engine on the server side is based on IfcOpenShell, which provides robust access to IFC semantics and geometry transformations.

A key implementation decision is the use of on-demand model access ("lazy loading") to limit memory usage: instead of keeping full IFC graphs in RAM, the service opens the IFC file when a specific operation is required and closes it afterwards. This keeps the backend lightweight and scales better for multi-project scenarios.

The backend also implements controlled model mutation logic for the mini-BIM/FM editor workflow. When the user modifies an object transform in the web client, the backend receives a 4×4 transformation matrix and translates it into an IFC-compatible placement update:

1. Decomposition of the client matrix into translation and rotation components.
2. Coordinate system alignment between the client's Y-up convention (Three.js) and the IFC's Z-up convention, ensuring correct axis mapping and sign consistency.
3. Serialization of the update back into the target element's IfcLocalPlacement / IfcAxis2Placement3D, preserving a valid IFC structure.

In addition, the backend reconstructs the spatial hierarchy (IfcProject → IfcSite → IfcBuilding → IfcBuildingStorey) by traversing IfcRelAggregates relationships. The result is served to the frontend as nested JSON to populate navigation trees and support storey-aware tools such as clipping and room-based analytics.

6.1.3 GIS services (footprint, campus layers)

To provide environmental context, GIS data is fetched dynamically rather than being bundled into the BIM model. The gis.py router queries OpenStreetMap via the Overpass API and returns GeoJSON suitable for browser rendering and 3D extrusion.

A commonly used pipeline is:

1. Receive site location (latitude/longitude) from user input or project metadata.
2. Construct a bounding box query (e.g., campus-scale radius).
3. Filter for relevant tags such as way["building"].
4. Convert response into GeoJSON and attach properties (e.g., height, roof attributes when available) to support 2D/3D visualization on the client.

This approach keeps the prototype fully open and reproducible while enabling scalable contextualization without requiring proprietary basemaps.

6.1.4 FM data models and database schema

The persistence layer is implemented as a hybrid prototype-oriented architecture. A relational database (SQLite via SQLModel, optionally extensible to PostgreSQL) stores structured FM entities and configuration metadata. At the same time, IFC project assets and sensor datasets are persisted in the filesystem as IFC/JSON artifacts.

- Relational storage (SQLite/SQLModel) is suitable for FM entities that require structured queries and updates, such as:
    - Notes and annotations linked to elements via IFC GlobalId.
    - Work orders and status tracking linked to a specific element or space.
- Filesystem/JSON storage supports:
    - IFC files and project packages.
    - sensor snapshots / time series exports when treated as lightweight datasets in a research prototype.

In the thesis, this can be framed explicitly as a deliberate choice: the prototype prioritizes transparency and fast iteration while remaining compatible with a future migration to a full relational/TSDB sensor backend.

6.1.5 REST and streaming endpoints

The backend exposes two interaction modes:

- Transactional REST endpoints (GET/POST/PATCH/DELETE) for CRUD operations, such as creating notes, work orders, assets, or managing projects.
- Near real-time streaming for sensor updates via Server-Sent Events (SSE). The /api/sensors/stream endpoint produces a continuous stream of JSON payloads using FastAPI's StreamingResponse. This design provides low-latency updates without the overhead of WebSockets and fits the "server pushes updates, client listens" model used for dashboards and digital twin monitoring.

6.2. Frontend implementation (Vite + web-ifc + three.js +Cesium)

The frontend is implemented as a web client built with Vite and modern JavaScript modules. The application integrates two rendering systems: CesiumJS for geospatial context (globe, terrain, coordinate reference) and Three.js for detailed BIM visualization and interaction. The BIM model is parsed in the browser with web-ifc (WebAssembly), enabling high-performance IFC access without server round-trips for every selection.

6.2.1 IFC 3D viewer and properties panel

The IFC viewer loads and renders the model using a web-ifc-based pipeline. The main goals are:

- Smooth interaction for large models.
- Element picking and highlighting.
- Property inspection for selected objects.

To keep interaction responsive, the viewer relies on efficient selection techniques. Instead of duplicating full geometry when highlighting, the client uses subset-based rendering strategies

(partial meshes) for selection and category visualization, which reduces GPU overhead and prevents unnecessary memory growth.

6.2.2 Georeferencing status and calibration helper

Because many real-world IFC/Revit exports are not correctly georeferenced, the client provides a clear georeferencing status and an optional calibration workflow. The Cesium globe establishes the global reference frame, while the BIM layer is transformed into place using project offsets or manual calibration when IfcSite reference coordinates are missing or unreliable.

A calibration UI allows the user to apply corrective offsets (e.g., UTM Easting/Northing or local translation), updating the BIM root transform without rebuilding the entire model. This is especially useful for campus-scale visual alignment when high-precision surveying is out of scope.

6.2.3 Category filters and view modes

The viewer supports semantic filtering by IFC type (e.g., walls, slabs, doors, windows, spaces, MEP elements). These features are implemented through client-side category indexing and selective rendering:

- Category filters enable showing/hiding element groups based on IFC classes.
- Technical / X-Ray mode (Ghost Mode) makes architectural layers semi-transparent to reveal internal systems (MEP/IoT), implemented through material changes (opacity adjustments and depth handling) while preserving navigation usability.

A dedicated manager component (e.g., a SystemsManager) centralizes state and prevents logic duplication across tools.

6.2.4 FM tools

FM-oriented inspection tools were implemented directly in the 3D client to reduce latency and improve usability:

- Clipping/section tools based on WebGL clipping planes support floor-by-floor inspection and internal visibility.
- Plan overlays can be attached as raster layers (e.g., PNG) mapped onto a 3D plane aligned with the section cut, supporting hybrid 2D/3D analysis where needed.

These tools are designed to be “operator-friendly”: one-click activation, clear UI state, and visual feedback consistent with professional viewers.

6.2.5 Mini BIM/FM editor

The prototype implements a minimal editing workflow that supports operational FM tasks such as placing, moving or rotating assets and linking them to BIM semantics.

On the client side, the editor uses transform gizmos (e.g., TransformControls) and a controlled event loop:

1. pointerdown: temporarily disables orbit camera controls to avoid interaction conflicts.

2. pointermove: updates a local transform preview in real time (client-side feedback).

3. pointerup: commits the change by sending a compact delta/transform payload to the backend.

On the backend, the change is validated and persisted either as FM entity updates (database) or as IFC placement modifications when the workflow explicitly mutates the BIM model. This separation allows the thesis to discuss two levels of editing: FM-level placement (non-destructive metadata) and IFC-level mutation (destructive but interoperable).

6.2.6 Sensor visualization and (near) real-time updates

The prototype supports near real-time sensor monitoring by combining a lightweight data model with a web-friendly streaming mechanism. On the backend, IoT data can be exposed in two ways: (i) request/response endpoints for querying the latest values or a short history window, and (ii) a Server-Sent Events (SSE) stream that continuously pushes new observations to connected clients.

On the frontend, a dedicated sensor visualization module subscribes to the stream and updates the UI without requiring page reloads. Sensor readings are rendered as interactive overlays (e.g., markers/billboards in the GIS view or 3D symbols in the BIM scene), optionally including a tooltip panel with the most recent value, unit, timestamp, and sensor identifier.

A key design goal is keeping the sensor-to-BIM relationship explicit: each sensor record can reference an IFC anchor (e.g., IFC GlobalId / ExpressID or a spatial container such as IfcSpace), enabling users to locate the sensor contextually in the 3D model and to support FM workflows such as inspection, diagnostics, and reporting. For development and reproducibility, the system supports both real sensors and replay datasets.

6.3 Deployment and open-source repository

The project is organized as an open-source codebase with separate backend and frontend repositories (or modular packages). The implementation prioritizes reproducibility: a new user should be able to run the prototype locally with minimal setup.

Planned or partial deployment components include:

- Containerization (Docker) to build and ship consistent environments.
- Service orchestration (docker-compose) to coordinate frontend, backend, and persistent volumes.
- Repository hygiene: clear README instructions, environment templates, and consistent folder structure for data/ artifacts and sample projects.

Even when running in development mode, these practices support a smooth transition from prototype to a shareable research artifact, aligned with open science and long-term maintainability.

7 Case Study and Evaluation

This chapter demonstrates and evaluates the proposed IFC Viewer & Facility Management system as a functional prototype. Building on the architecture and requirements defined in Chapters 5 and 6, the evaluation is conducted through scenario-based workflows that reflect typical facility management interactions in a web environment. The goal is to provide clear evidence that the prototype achieves end-to-end integration across BIM (IFC), GIS context, IoT monitoring, and FM operations, while remaining lightweight and reproducible.

To keep the evaluation readable, the implemented functionality is presented in the main chapter as a set of facility-management use cases, each described through: (i) a short goal statement, (ii) summarized user steps, (iii) expected system behavior, and (iv) visual evidence via screenshots. At the same time, reproducibility is ensured by providing a complete step-by-step workflow catalogue (W1–W23) in Appendix A, which documents the full operational procedures in detail.

The remainder of this chapter is structured as follows: Section 7.1 describes the evaluation setup and datasets. Section 7.2 introduces the grouping of workflows into use cases. Sections 7.3–7.8 present the use-case demonstrations and results, supported by annotated figures. Finally, the chapter concludes with a short summary of requirement coverage and key observations relevant to prototype limitations and future extensions.

7.1 Evaluation setup

The prototype was executed in a local, on-premise setup consisting of a web client and a backend API. IFC models were used as BIM input datasets, contextual GIS layers were loaded on demand, and IoT sensor values were visualized via near real-time updates. Evidence was collected as annotated screenshots for each use case and the full workflow procedures are reported in Appendix A.

7.2 Defined Facility Management (FM) Use-Cases

To ensure completeness while maintaining readability, the 23 implemented workflows are organized into six overarching Facility Management (FM) use cases. This grouping demonstrates how granular technical actions—such as parsing geometry or fetching API endpoints—combine to solve real-world operational challenges within the built environment. A detailed, step-by-step catalog of all individual workflows is provided in Appendix A.

To ensure a comprehensive evaluation, all 23 implemented workflows and tools were distributed across the defined use cases.

- Project & Model Management: Incorporates Project Administration (Tool 22), Structure Tree navigation (Tool 9), Internationalization (Tool 23), and Asset Management (Tool 14).
- Spatial & Contextual Alignment: Integrates Hybrid Georeferencing (Tool 1), 3D Context GIS Environment (Tool 12), Global Environment with CesiumJS (Tool 13), and the GIM Geo-Footprint Layer (Tool 21).

- Technical Inspection & Coordination: Groups the Systems Analysis/Tech Mode (Tool 7), Advanced Coordination and Clash Detection (Tool 5), Section Box clipping (Tool 15), View Cube navigation (Tool 11), and Floor Plans management (Tool 6).

- Interactive Analysis & BIM Editing: Encompasses the core Selection & Properties (Tool 10), Box Selection (Tool 20), Advanced Search (Tool 19), Visual Controls (Tool 18), Measurements (Tool 2), and the BIM Editor CRUD operations (Tool 3).

- Facility Operations: Evaluates the Facility Management Work Orders (Tool 8), Annotations (Tool 17), Quantity Take-Off reports (Tool 16), and the Real-time IoT Module (Tool 4).

7.2.1. Model Management & Spatial Context

This use case encompasses the foundational steps of establishing the digital twin environment. It involves uploading raw IFC files to the server and federating multiple localized BIM disciplines. To navigate this complex data, the system relies on the Spatial Navigation Tree (IFC Tree), allowing users to drill down from the site level to individual building storeys and isolated elements. The frontend handles the rendering of this hierarchy dynamically, ensuring a lightweight client experience even with complex federated models.

To execute this use case, several core tools were developed and evaluated. Project Administration (Tool 22) handles the initial file uploads and database creation. Once loaded, the Structure Tree (Tool 9) allows hierarchical navigation of the IFC file, while Asset Management (Tool 14) permits the injection of external GLB models. The entire interface is supported by an Internationalization module (Tool 23) for multilingual accessibility.

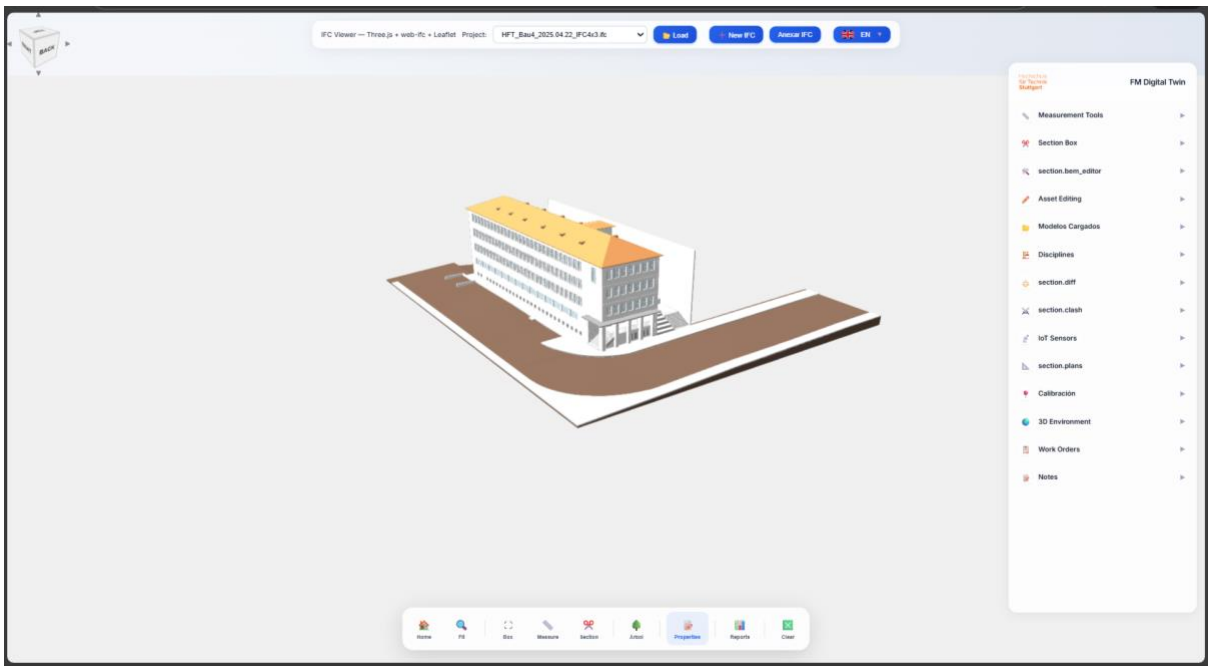

Figure 3 Project Administration (Tool 22) Internationalization module (Tool 23)

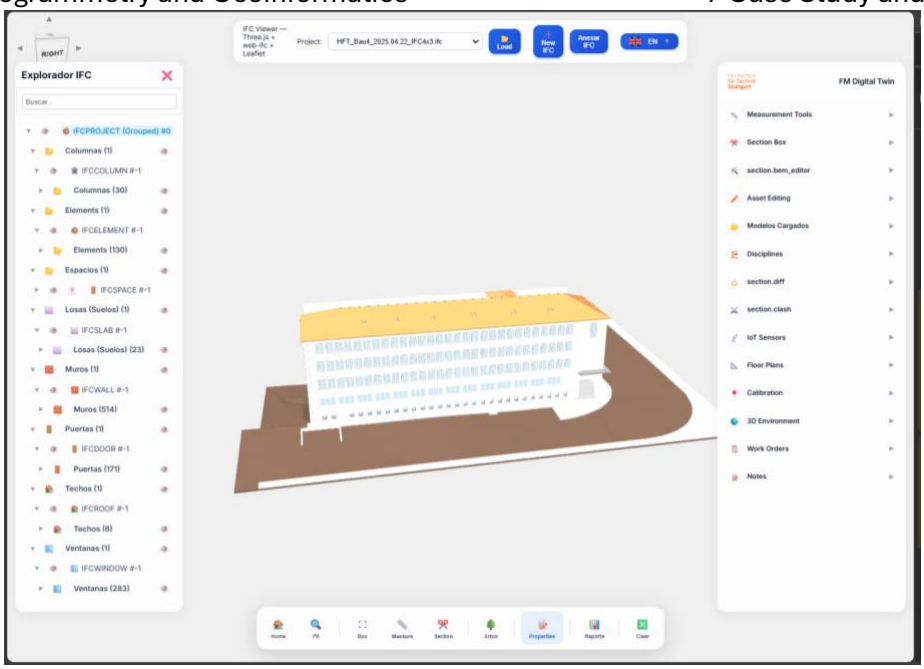

Figure 4 the Structure Tree (Tool 9)

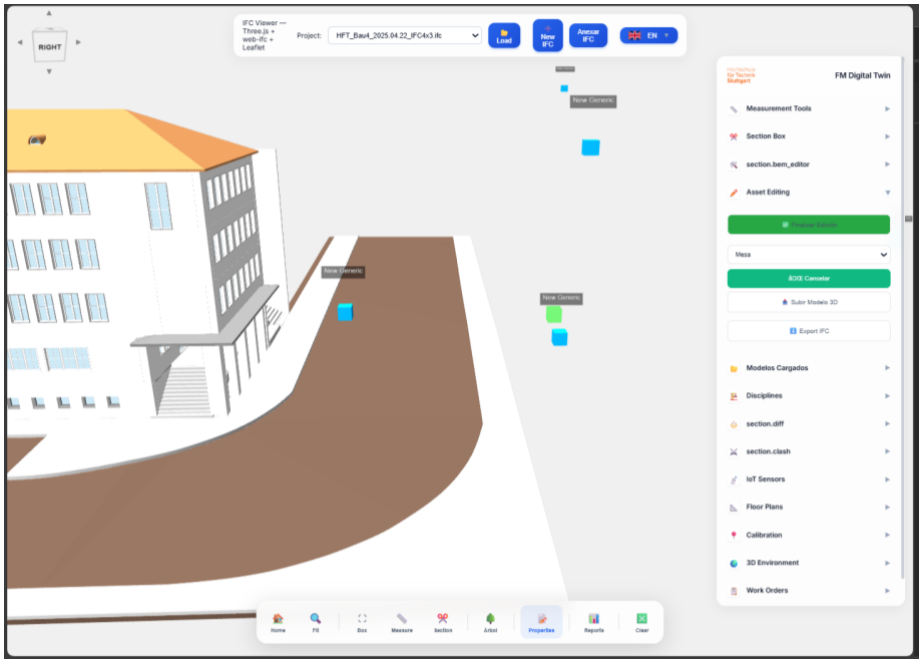

Figure 5 Asset Management (Tool 14)

7.2.2 Geospatial Alignment & Calibration

Standard IFC viewers often render buildings in an empty coordinate space. This use case resolves that limitation by anchoring the BIM models to real-world coordinates. Workflows within this group include toggling 2D GIS footprint overlays (GeoJSON) onto the mapping context and performing manual or automated geospatial calibrations. By bridging CesiumJS for the global coordinate system (ECEF) and Three.js for local rendering, the user can verify the building's correct positioning against external geographic data without altering the source file in the browser.

This spatial alignment relies on a combination of specific visualization tools. The Global 3D Environment (Tool 13) initializes CesiumJS in the background. To validate the alignment, users can toggle the GIM Layer (Tool 21) for 2D cadastral footprints.

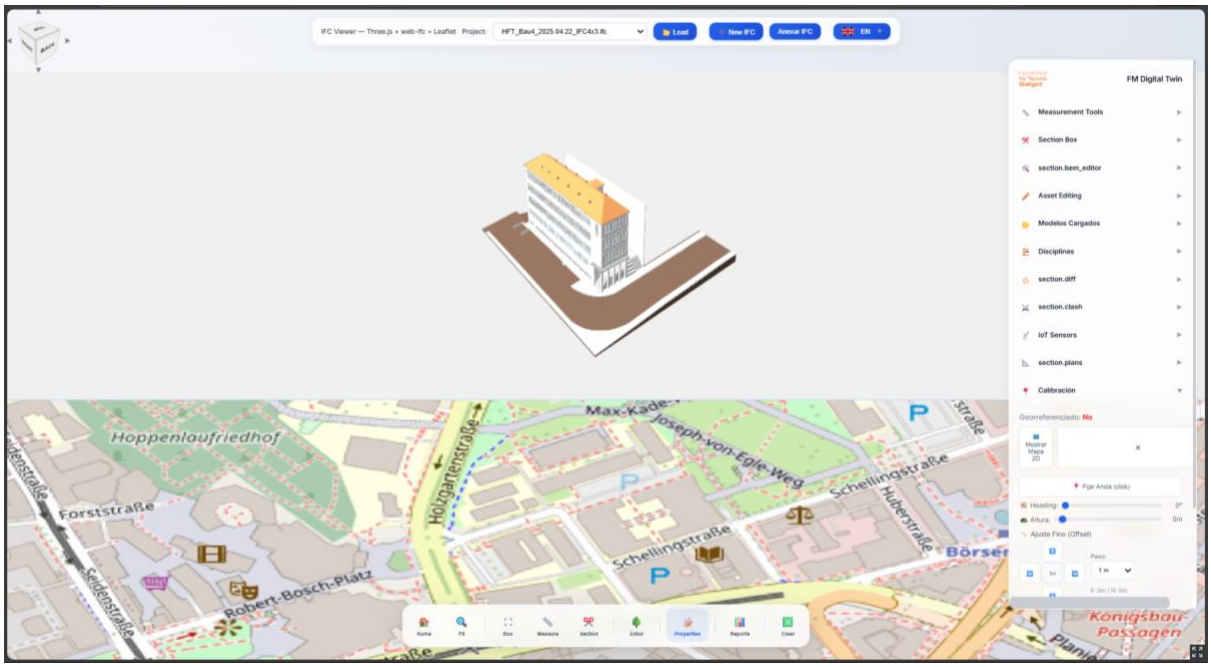

Figure 6 Georeferencing (Tool 21)

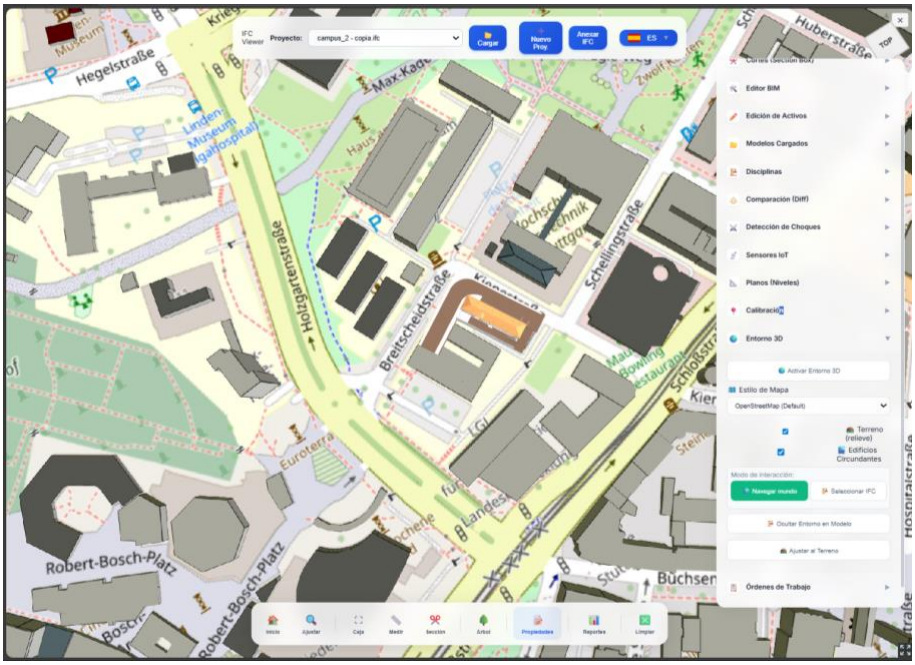

Figure 7 The Global 3D Environment (Tool 13)

7.2.3 Facility Operations & Reporting

Transitioning from visualization to active management, this use case covers the daily administrative tasks of facility managers. It groups the workflows for creating and managing Work Orders, attaching persistent notes to specific IFC elements via their GlobalId, and generating Quantity Takeoff (QTO) reports. Data persistence relies on the relational backend

database, linking the 3D element's spatial context directly to operational tickets via the /work-orders/ and /ifc/notes/ REST endpoints.

Facility managers utilize a specific set of interactive features for daily operations. Facility Management (Tool 8) links database work orders directly to 3D elements, supplemented by Annotations (Tool 17) which project 3D HTML notes onto the geometry. For administrative auditing, the Quantity Take-Off (Tool 16) dynamically aggregates and exports material data (volumes, areas) from the selected elements.

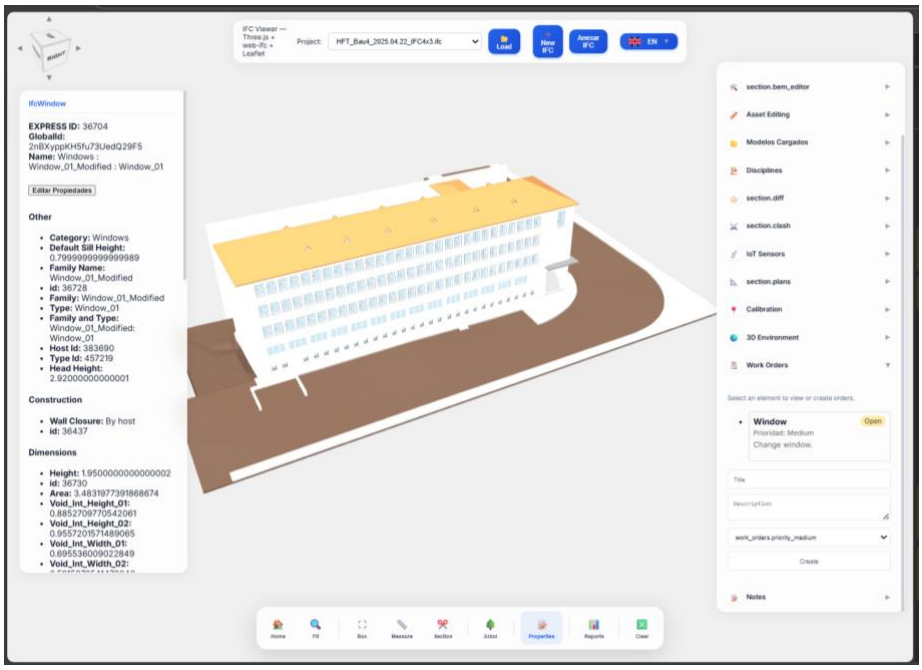

Figure 8 Annotations (Tool 17)

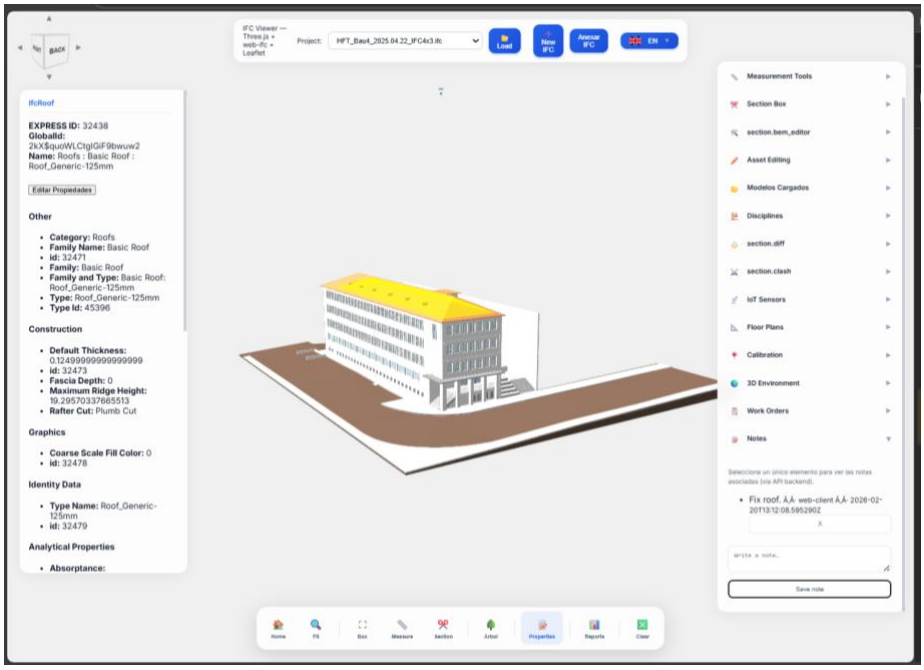

Figure 9 Facility Management (Tool 8)

7.2.4 Real-Time IoT Monitoring

To evaluate the system's capability as a dynamic digital twin, this use case tests the integration of live sensor data. It includes workflows for visualizing sensor locations within the 3D space and streaming data via Server-Sent Events (SSE). By handling the continuous stream of data (/sensors/stream) efficiently, the frontend dynamically updates visual indicators, such as temperature heatmaps, in real-time without requiring scene re-renders or suffering performance degradation.

This dynamic monitoring is exclusively driven by the IoT Module (Tool 4). By utilizing Server-Sent Events (SSE), the frontend dynamically updates the color matrices of IfcSpace geometries to create real-time heatmaps, proving the system's capacity to handle high-frequency data streams without re-rendering the entire scene.

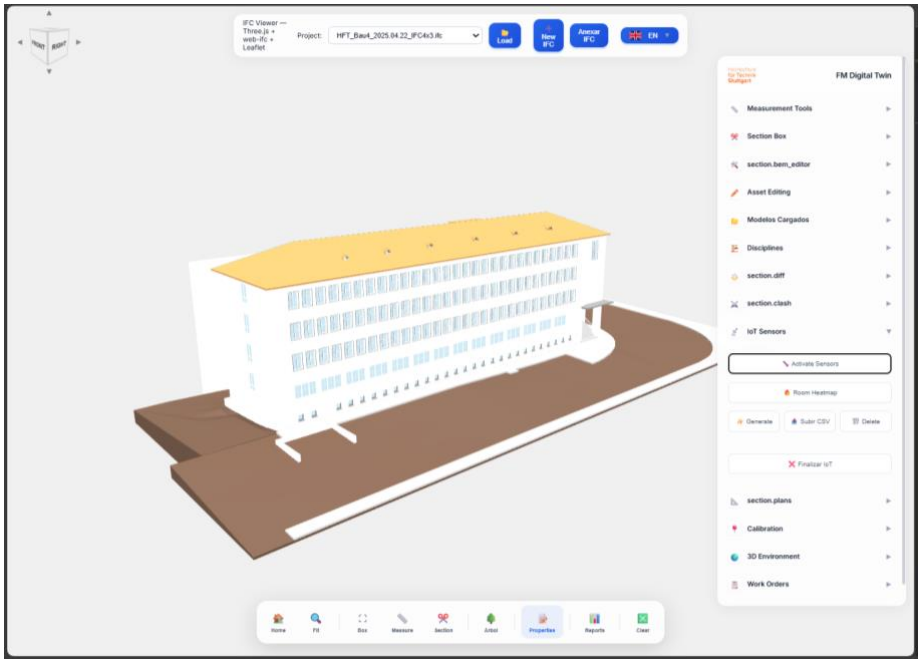

Figure 10 IoT Module (Tool 4)

7.2.5 Advanced BIM Coordination

This use case tests the system's capacity for in-depth technical review. Workflows grouped here include activating technical visual modes, utilizing the Section Box for cross-sectional internal inspections, and performing spatial measurements. These tools allow engineers to isolate specific MEP systems or structural anomalies.

Engineering coordination is achieved through specialized viewing modes. The Systems Analysis / Tech Mode (Tool 7) renders architectural elements transparent to expose MEP systems. Users can further isolate issues using the Section Box (Tool 15) for dynamic clipping, navigate efficiently using the View Cube (Tool 11), or switch to traditional 2D overlays using the Floor Plans tool (Tool 6). Finally, Advanced Coordination (Tool 5) calculates bounding-box intersections to highlight clashes between federated models.

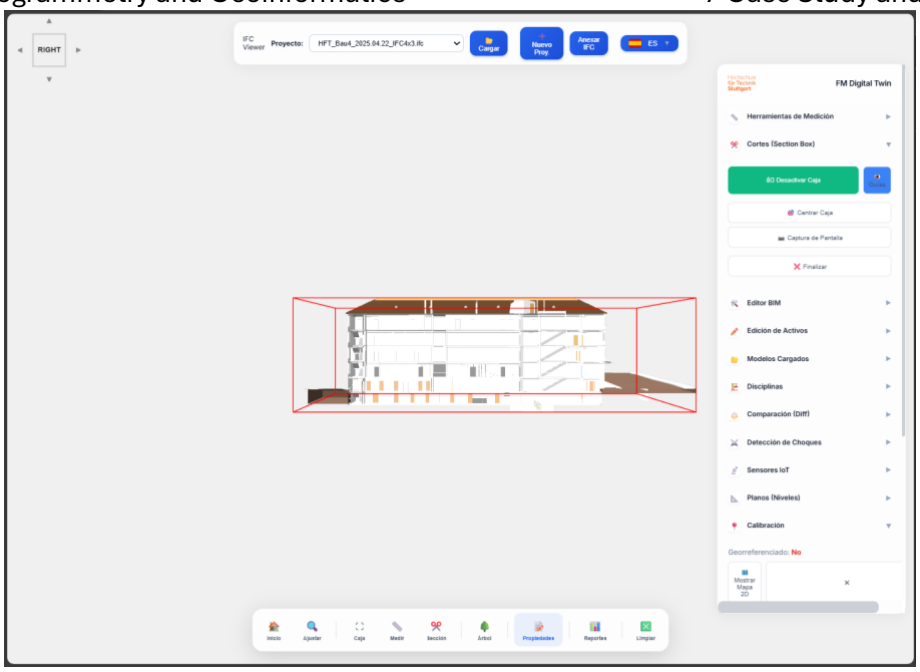

Figure 11 Section Box (Tool 15)

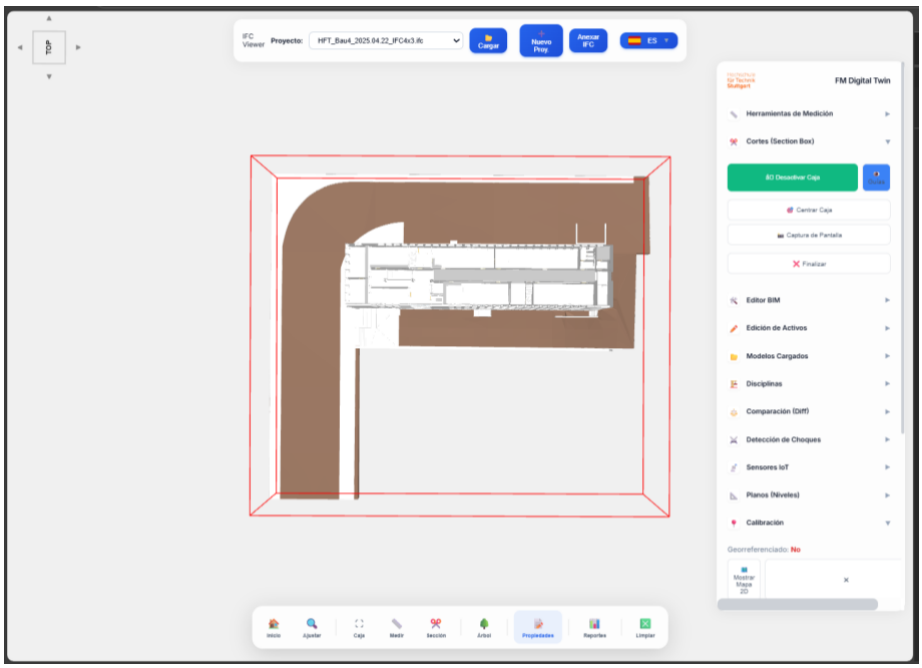

Figure 12 Section Box (Tool 15)

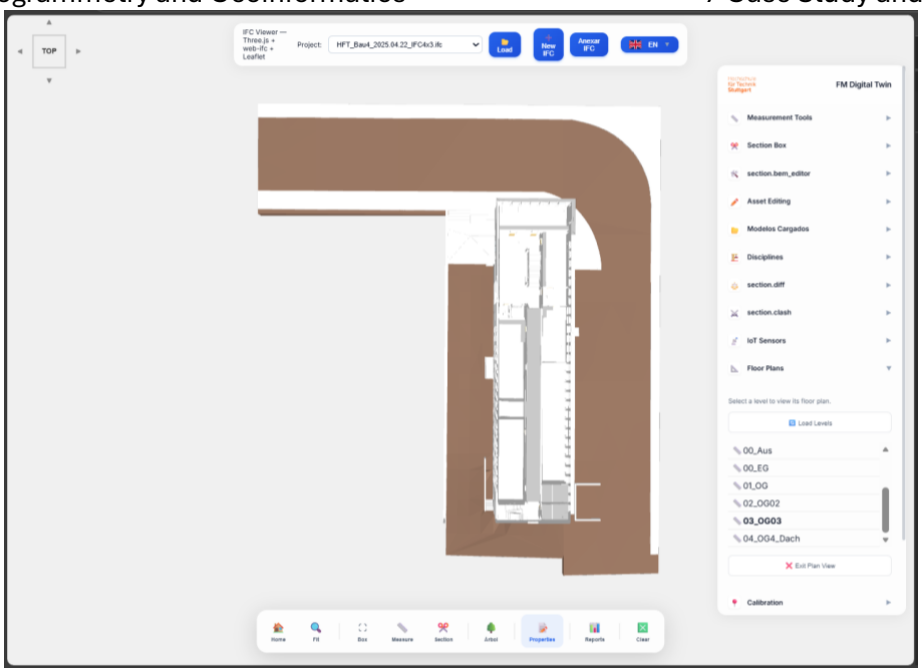

Figure 13 Floor Plans tool (Tool 6)

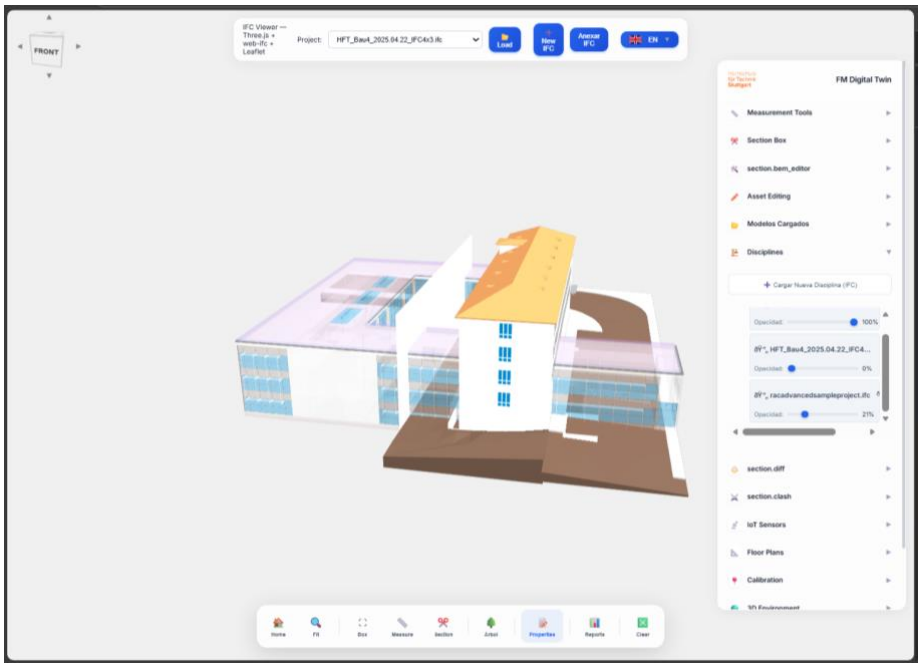

Figure 14 The Systems Analysis / Tech Mode (Tool 7)

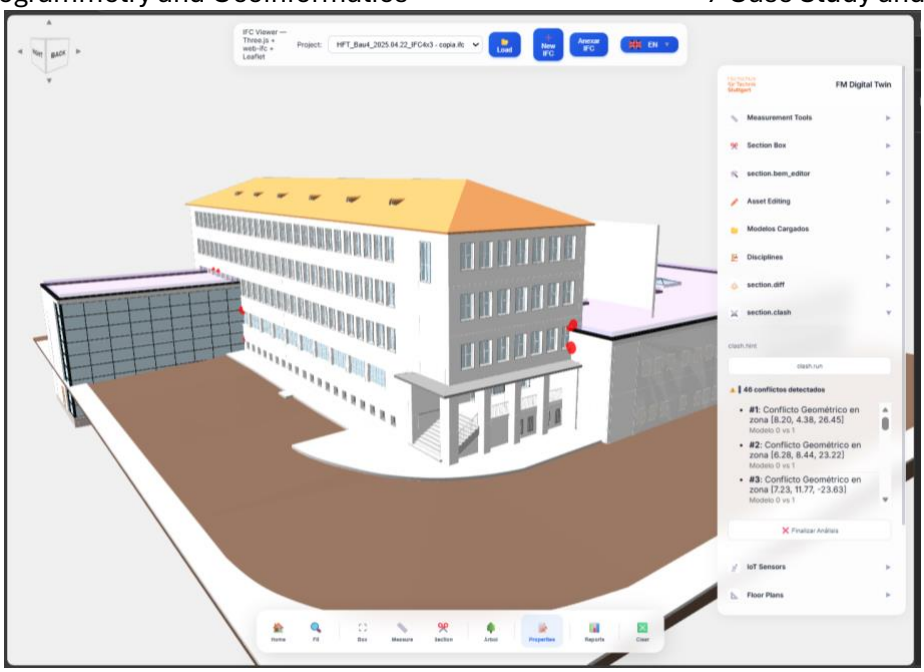

Figure 15 Advanced Coordination (Tool 5)

7.2.6 Interactive Inspection & Modification

The final use case evaluates the system's ability to act as a bidirectional platform rather than a static viewer. It groups workflows related to direct element selection, viewing hypermedia properties, and utilizing the BIM Editor to modify asset coordinates. Crucially, any spatial transformation or mathematical modification applied to an element is securely transmitted to the ifc_editor.py backend. IfcOpenShell processes the structural mutation of the .ifc file on the server, while the frontend strictly updates the visual state, preserving data integrity.

This use case involves the most direct interaction with the geometry. Users can locate elements via Advanced Search (Tool 19), isolate them using Box Selection (Tool 20) or standard Selection & Properties (Tool 10), and quantify them with the Measurements tool (Tool 2). Crucially, any physical changes made via the BIM Editor (Tool 3) adhere to the strict architectural rule: the frontend exclusively handles the visual Gizmos, while the backend's IfcOpenShell strictly processes the mathematical coordinate translations and writes the .ifc file.

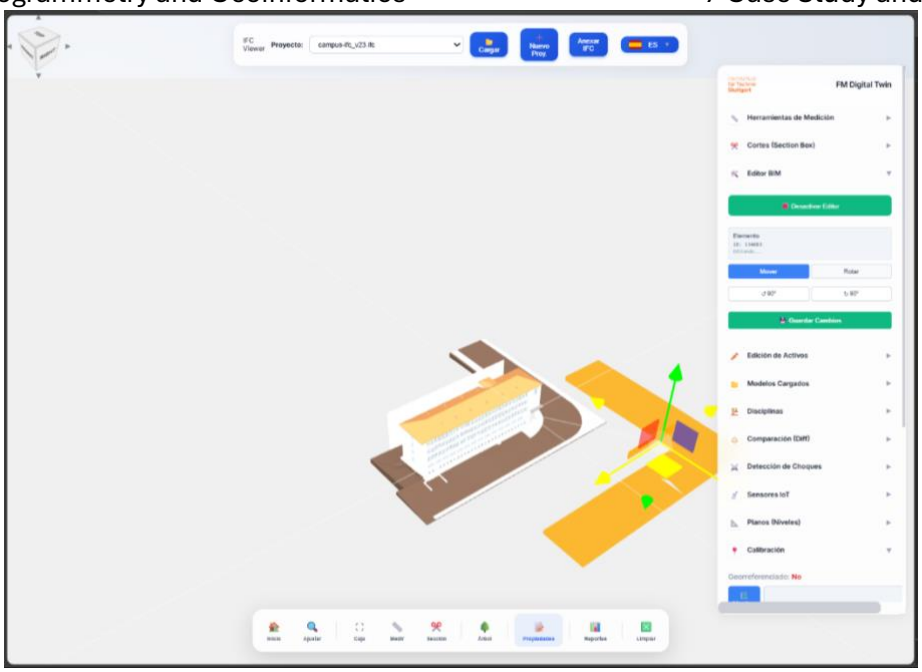

Figure 16 the BIM Editor (Tool 3)

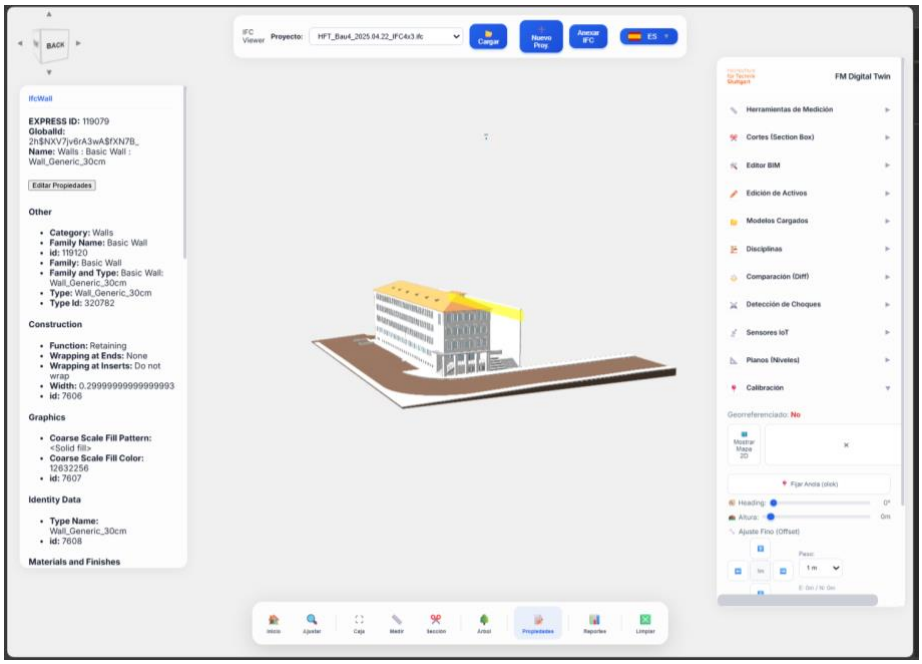

Figure 17 Selection & Properties (Tool 10)

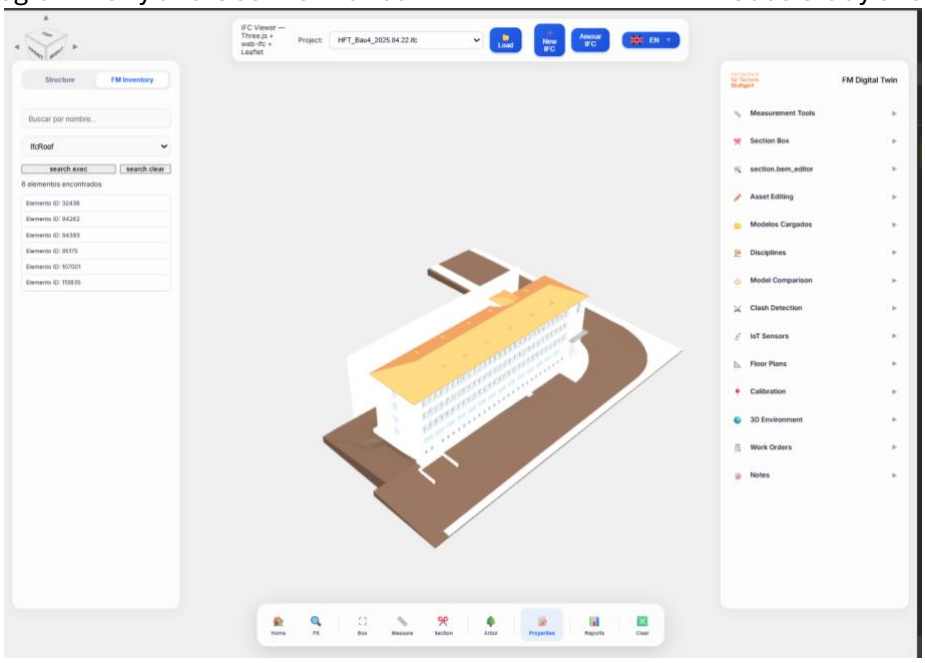

Figure 18 Advanced Search (Tool 19)

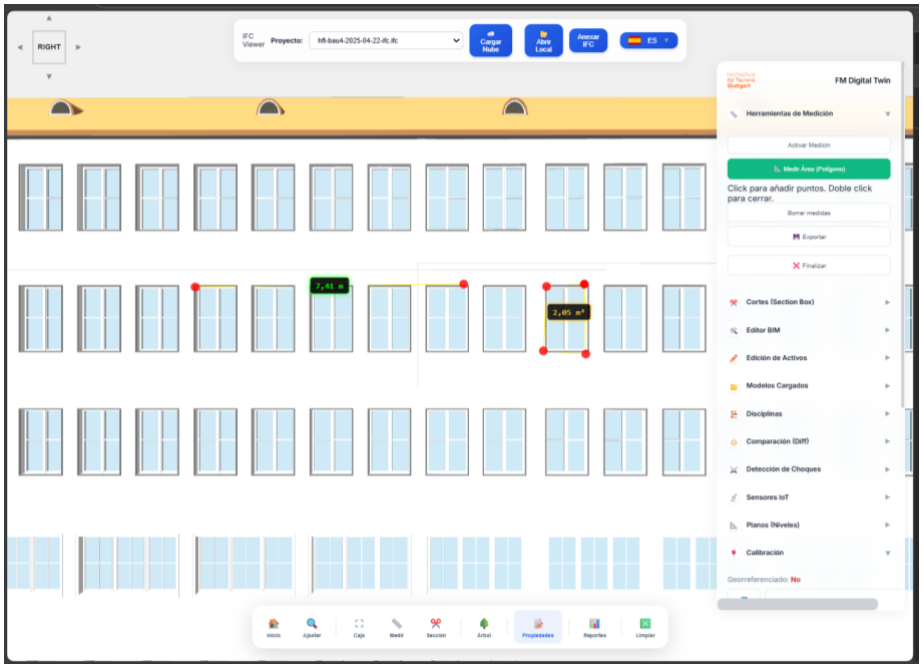

Figure19 Measurements tool (Tool 2)

7.3 Functional evaluation

The functional evaluation of the proposed framework was conducted using the HFT Stuttgart campus dataset. The primary objective was to verify the system's responsiveness, accuracy, and overall stability when executing the defined Facility Management use cases in a standard web browser environment.

The evaluation confirmed that the architectural separation of concerns—strictly isolating visualization tasks on the client side from complex processing on the server side—was highly effective.

- Model Management and Rendering Efficiency: During the upload and federation of multiple IFC models (Architecture, MEP, Structure), the system maintained high performance. The backend successfully handled the file parsing and initialization (POST /sites/upload), allowing the frontend to utilize web-ifc strictly for generating lightweight WebGL geometry. The browser maintained a stable frame rate even when toggling visibility across federated disciplines.

- Geospatial and Contextual Accuracy: The integration of CesiumJS (global ECEF coordinates) with Three.js (local Cartesian coordinates) functioned as intended. The loading of 2D GIS footprint overlays via GeoJSON provided immediate contextual validation for the 3D models. Manual calibration tools accurately shifted the local scene over the global map without requiring the original IFC file to be regenerated, proving the efficiency of hybrid coordinate rendering.

- Operational Data Linkage: The linkage between 3D spatial elements and relational data (Facility Operations) proved robust. By using the IFC GlobalId as the primary key, the frontend successfully retrieved and displayed Work Orders and attached notes from the backend database via REST API calls (/work-orders/, /ifc/notes/). Generating Quantity Takeoff (QTO) reports successfully extracted spatial properties dynamically, verifying the integrity of the data bridging.

- Real-Time Data Handling (IoT): The system successfully handled continuous data streams for the IoT Monitoring use case. Utilizing Server-Sent Events (SSE) through the /sensors/stream endpoint allowed the frontend to receive asynchronous updates. Visual indicators, such as color-coded heatmaps and sensor markers, updated smoothly without triggering full scene re-renders, demonstrating the framework's capability as a live digital twin platform.

- Data Integrity during Modifications: The most critical functional test involved the interactive modification of the BIM model, such as moving assets. The system strictly enforced its processing hierarchy: the client-side BIM Editor captured the user's intent and new coordinates, but the actual structural mutation of the file was executed exclusively by IfcOpenShell on the server (/ifc_editor.py). This approach prevented client-side memory overloads and guaranteed that the resulting .ifc file remained standardized and uncorrupted.

Overall, the functional evaluation demonstrated that the system successfully fulfills the requirements of a modern, web-based Facility Management tool, balancing high-fidelity 3D/GIS visualization with secure, server-side data manipulation.

During the functional evaluation of the interactive tools (specifically Tool 3: BIM Editor and Tool 2: Measurements), the architectural division of tasks proved crucial for system stability. By strictly maintaining that the backend (via IfcOpenShell) processes all structural and mathematical modifications, the frontend successfully remained a lightweight visualization layer. When users manipulated elements using the BIM Editor, the frontend only handled the UI gizmos, while the actual geometry rewriting occurred safely on the server.

Furthermore, the evaluation of large-scale querying tools—such as Advanced Search (Tool 19), QTO Reports (Tool 16), and the Structure Tree (Tool 9)—demonstrated that client-side memory was effectively managed. Extracting properties and filtering the model tree performed

smoothly because the web-ifc engine in the frontend was only tasked with reading pre-parsed arrays, validating the robustness of the user interface even with complex federated models.

7.4 Technical observations

Throughout the development and evaluation phases, several technical observations were recorded regarding system performance, architectural bottlenecks, and data management.

- Client-Side Rendering Limits: Utilizing web-ifc in a WebAssembly (WASM) environment proved highly efficient for parsing raw geometry. However, rendering massive federated models natively in the browser's main thread can lead to memory saturation. The system mitigated this by implementing lazy loading and allowing users to toggle individual discipline visibilities, keeping the WebGL context lightweight.

- Unidirectional Data Streaming (IoT): For the real-time sensor integration, Server-Sent Events (SSE) were chosen over WebSockets (/sensors/stream). This observation confirmed that for telemetry data—where the flow is predominantly server-to-client—SSE provides a lighter, HTTP-native alternative that seamlessly integrates with FastAPI without the overhead of maintaining bi-directional socket states.

- Hybrid Persistence Strategy: The evaluation validated the hybrid database approach. Storing dynamic Facility Management data (Work Orders, Notes, Asset properties) in a relational SQLite database while keeping the .ifc file intact as a static source of truth prevented file corruption. The linkage via the IFC GlobalId acted as a reliable primary key across the SQL and NoSQL domains.

- Server-Side Mutation: A critical observation was the computational cost of editing IFC files. Attempting to modify element coordinates or metadata purely in the browser risks data loss. Delegating all structural file mutations to the IfcOpenShell engine via the /ifc_editor.py endpoint ensured that the output files strictly adhered to the STEP physical file format standards, albeit requiring a brief asynchronous loading state on the client.

- Dual-Engine Synchronization: A significant technical observation during the evaluation of Tools 1, 12, and 13 was the performance of the synchronization loop between Three.js and CesiumJS. Reading the Cesium Camera State (Heading, Pitch, Roll, Position) and cloning it to the Three.js camera on every animation frame maintained a rigid visual attachment of the BIM model to the Earth. However, it was observed that extremely high-poly IFC models could cause minor frame-rate drops during rapid camera panning due to the simultaneous rendering demands of the local WebGL context and the global ECEF terrain map.

7.5 Discussion

The implementation and evaluation of this web-based framework highlight significant ongoing challenges and opportunities within the intersection of BIM, GIS, and Facility Management.

- The Georeferencing Gap in Standard IFC: One of the most prominent challenges encountered was the inherent lack of explicit, standardized geospatial data (like EPSG codes) in typical IFC exports from authoring tools like Revit. Buildings are usually modeled in a local, Cartesian void. This research demonstrates that a hybrid

visualization approach—using CesiumJS to establish the global ECEF context and Three.js to render the local matrix—effectively bridges this gap. It allows facility managers to visually align and validate a building against GIS footprints (GeoJSON) without requiring complex, pre-processing georeferencing scripts.

- Overcoming Data Silos: Traditional FM relies on CMMS (Computerized Maintenance Management Systems) that are disconnected from spatial data. This framework proves that using the IFC GlobalId as a universal identifier breaks down these silos. By coupling a 3D element directly to REST API endpoints, a digital twin becomes an active operational dashboard rather than a passive 3D viewer.

- Future Outlook: While the current MVP successfully demonstrates a bidirectional flow for basic coordinate editing and data linkage, future iterations could explore integrating point cloud data (as originally scoped in the broader research context) directly into the web viewer. Additionally, transitioning the backend to a more robust PostgreSQL instance would be necessary to handle concurrent, multi-user facility management operations at a city-scale level.

8 Conclusions and Future Work

8.1 Summary of contributions

The culmination of this research represents a substantial advancement in resolving information fragmentation within the Facility Management (FM) industry. Historically, the Operations and Maintenance (O&M) phase—which constitutes the longest and most costly period in any facility's lifecycle—has relied on disparate software systems. Traditional Computerized Maintenance Management Systems (CMMS) and Computer-Aided Facility Management (CAFM) platforms have provided mature administrative capabilities, but have systematically failed to use Building Information Modeling (BIM) models as primary operational databases, relegating them to mere passive visual repositories or single-use tools during handover. Likewise, the integration of geospatial context (GIS) and dynamic building telemetry (IoT) has required proprietary and expensive platforms, inducing severe vendor lock-in and hindering the true formation of Digital Twins.

To bridge this gap, this thesis has designed, implemented, and evaluated an open-source, modular framework based entirely on web technologies, structured through a *Design Science Research* (DSR) methodology. This Minimum Viable Product (MVP) successfully demonstrates the feasibility of unifying heterogeneous data sources into a fluid workflow accessible from a standard browser, without requiring proprietary plugins or commercial software licenses.

The proposed system's architecture is based on the Industry Foundation Classes (IFC) standard (ISO 16739), using it not only as a geometric representation schema but as the central semantic backbone. The most significant technical contribution lies in the orchestration of a clearly defined three-tier architecture, designed to separate visualization, business logic, and persistence concerns:

Architectural Layer	**Implemented Technological Components**	**Primary Function and Contribution**
Presentation Layer (Frontend)	Vite, Vanilla JS/Vue, Three.js, CesiumJS, web-ifc (WASM).	Orchestrates a dual (hybrid) graphic engine that synchronizes local BIM geometry (Three.js) with the global geospatial environment (CesiumJS). Accelerates IFC parsing via WebAssembly to ensure fluidity in massive models.
Application Layer (Backend)	FastAPI (Python 3.10+), IfcOpenShell.	Acts as an IoT mediator and mathematical transformation engine. Exposes a REST API based on hypermedia principles (HATEOAS), ensuring that CRUD

Architectural Layer	Implemented Technological Components	Primary Function and Contribution
		operations on the IFC model are explicit and traceable.
Data Layer (Storage)	SQLite / PostgreSQL, Local File System (JSON, .ifc).	Implements a hybrid persistence strategy. Structured metadata (work orders, notes) is housed in relational databases, while geometry is preserved intact in the original IFC files.

Table 2 Architecture Contributions

Another high-impact technical contribution developed in this framework is the implementation of a high-performance Clash Detection system optimized for direct client-side execution. Surpassing simple Axis-Aligned Bounding Box (AABB) checks, this system integrates Bounding Volume Hierarchies (BVH) through the three-mesh-bvh library. It incorporates rigorous geometric validation using bidirectional raycasting algorithms, drastically reducing false positives, and applies spatial clustering algorithms to offer a clean visualization. Furthermore, it consolidates a BIM audit workflow by exporting technical reports in CSV format linked to spatial identifiers.

This research has empirically validated the viability of this architecture through the development of 23 functional workflows tested in a university case study environment (HFT Stuttgart campus). By anchoring FM operational entities to the model's standardized topology, the research demonstrates that an open-source ecosystem can rival commercial solutions in orchestrating a bidirectional Digital Twin.

A highly significant technical contribution developed within this framework is the implementation of a high-performance Clash Detection system optimized for direct client-side execution. Surpassing standard Axis-Aligned Bounding Box (AABB) checks, this system integrates Bounding Volume Hierarchies (BVH) through the three-mesh-bvh library and utilizes bidirectional raycasting algorithms to rigorously validate true geometric intersections, drastically reducing false positives. Furthermore, the system implements a spatial clustering algorithm to group collision points for cleaner visualization and incorporates a BIM Audit workflow capable of exporting technical reports in CSV format, linking each conflict with its respective model identifier to facilitate traceability.

8.2 Answers to research questions

The exhaustive evaluation of the prototype has generated functional data and technical observations that provide a rigorous and well-founded answer to the four research questions formulated at the beginning of this study.

8.2.1 Research Question 1 (RQ1): BIM/FM Data Linking

Question: *How can an open-source, web-based framework use IFC semantics and persistent IFC GlobalId (GUID) identifiers to reliably link Facility Management (FM) records to specific BIM elements in the browser?*

The fundamental challenge of BIM-FM interoperability is avoiding data corruption and ensuring that a physical component's operational history survives geometric updates to the 3D model. The developed framework resolves this issue by establishing the unique GlobalId (a persistent 22-character alphanumeric hash defined in the IFC schema) as the absolute Primary Key crossing graphical and relational data domains.

Technically, the linking process is executed in a highly optimized sequence:

1. WebGL Interaction: When a user clicks on the 3D canvas, the Three.js engine uses *Raycasting* to project a vector detecting intersections with the polygonal mesh.

2. Semantic Isolation and Extraction: The web-ifc parser, operating at near-native speeds thanks to WebAssembly (WASM), identifies the internal ExpressID of the intersected element and extracts its immutable GlobalId.

3. Relational Bridge (REST API): With the GlobalId secured, the frontend communicates with the FastAPI backend. The underlying database hosts tables for entities (Work Orders, Notes) linked directly to this GlobalId via foreign keys.

4. Interface Hydration: Upon retrieving this information, the system projects interactive 3D markers fixed exactly at the element's geometric centroid, closing the visual loop.

8.2.2 Research Question 2 (RQ2): BIM-GIS Integration and Placement

Question: How can an IFC model be integrated with geospatial context in CesiumJS, and what lightweight workflow enables practical placement/georeferencing of IFC models when native georeferencing is missing or unreliable?

To achieve integration without destroying the high precision of the local architectural model, the thesis implemented a Dual Scene Hybrid Visualization. The system orchestrates two overlapping rendering loops: the CesiumJS engine renders the globe using the WGS84 ECEF system, while Three.js renders the detailed BIM model retaining its origin Cartesian coordinate matrix.

The optical illusion of a "single world" is achieved through a synchronization loop in every animation frame that extracts the Cesium camera state and clones it into the Three.js camera. Additionally, to mitigate the critical void of models exported without native georeferencing, the architecture introduced a Lightweight Geospatial Calibration Workflow. This flow allows introducing manual visual offsets, executing a "Snap to Terrain" command that queries true absolute elevation, and contextualizing the urban scale environment by consuming the Overpass API to extract OpenStreetMap (OSM) topological footprints.

8.2.3 Research Question 3 (RQ3): IoT Integration in an FM Prototype

Question: How can real-time IoT sensor streams be integrated into the same web application and associated with BIM components to support environmental monitoring and operational decision-making?

The architecture addresses this challenge by adopting design philosophies from the OGC's SensorThings API. To achieve this integration, the backend acts as a spatial brain performing pre-calculated Point-in-Polygon checks, identifying in real-time if a sensor's 3D coordinates fall within a room's volume (IfcSpace).

The system dispatches live telemetry to the client using Server-Sent Events (SSE), maintaining an optimized, low-latency channel ideal for server-to-client information flows. To process visual updates without crashing the browser, the system avoids full re-rendering. Instead, it injects the normalized value directly into a WebGL shader uniform variable, producing dynamic volumetric heatmaps in real-time with marginal computational cost.

8.2.4 Research Question 4 (RQ4): Validation and Feasibility

Question: To what extent does the proposed prototype support day-to-day FM tasks through CRUD operations, and how does it perform under realistic usage scenarios?

The empirical validation overwhelmingly demonstrated its practical feasibility. The system successfully and stably processes bidirectional CRUD operations. The BIM Editor allowed physical alteration of components in space; the brilliance lay in delegating responsibilities: the frontend only handles the visual interface and sends a Delta vector, while the server, equipped with IfcOpenShell, recalculates the transformation matrix and safely overwrites the physical file, protecting data integrity.

Performance against dense models was exceptionally agile thanks to the WebAssembly (WASM) parser, and memory management remained stable through Smart Grouping algorithms in the hierarchical DOM tree. Likewise, IoT transmission latency proved negligible, confirming the platform is mechanically ready for professional operations.

8.3 Implications for Facility Management practice

The findings of this project transcend the strictly academic realm, injecting substantial practical implications into the operational economy of professional Facility Management:

1. Eradication of Information Silos: By topologically anchoring administrative data, 3D modeling, 2D maps, and IoT sensors under a single IFC semantic ontology universally accessible from the web browser, information asymmetries collapse.

2. Data Sovereignty and Long-Term Interoperability (Vendor Lock-in): The commitment to 100% open-source ecosystems (SQLite, FastAPI, Three.js, CesiumJS, IfcOpenShell) guarantees a viable *On-Premise* implementation. Organizations retain full ownership and auditability of their own infrastructure intelligence without suffering economic licensing penalties.

3. Transition from Reactive to Predictive Maintenance: By establishing a physical and dynamic nexus between the asset's topology and its temporal thermodynamic

variables, the system catalyzes the transition towards predictive strategies driven by data analytics.

8.4 Limitations of the Study

Scientific objectivity imposes the need to contextualize the empirical scope of the research by detailing the technological restrictions and current system limitations faced:

- Case Study Data Asymmetry: There was a topographical decoupling between the BIM model and live telemetry. The truthful sensor data came from a different institutional building than the federated IFC file available for visual demonstration, requiring the use of a "Replay Mode".
- Ontological Rigidity of the IFC Schema for Temporal Data: The IFC standard is not natively conceived to agglutinate high-frequency time-series data. The effort to inject live telemetry had to be delegated to the middleware (backend + relational database), forcing a conceptual pairing.
- BIM Editor Stability (Proof of Concept Status): As of the current version, the BIM Editor remains in a PoC state with several known issues requiring refinement. While geometric changes are calculated on the client, the synchronization flow occasionally fails to rewrite the .ifc file accurately due to coordinate system mismatches (Y-Up vs Z-Up complexity). Furthermore, the interaction between TransformControls and the global scene selection often causes "ghosting" or accidental selection of background elements. Fine-grained alignment and "snapping" to existing geometry are not yet implemented, making precise architectural modifications difficult.
- Rendering Performance on Massive Datasets: Handling extremely large federated IFC models (>500MB) can lead to significant frame rate drops in the browser. The physical ceiling associated with the web browser's main thread memory, combined with the incessant camera synchronization loop with CesiumJS, restricts fluidity. Further optimization using geometry instancing and advanced occlusion culling is required.

8.5 Future work

Relying on the success of the current prototype and acknowledging its technical limitations, strategic directions for future research and development have been identified to evolve this framework into a production-ready Digital Twin ecosystem.

8.5.1 Robust BIM Authoring

Bi-directional Synchronization: The transformation engine must be rebuilt to allow seamless, error-free updates between the Three.js viewport and the persistent IFC database, resolving current coordinate system mismatches.

Component Library Integration: Future iterations should integrate a library of standardized BIM objects (furniture, equipment, sensors) that can be "drag-and-dropped" into the scene, featuring automatic GUID generation and semantic metadata assignment.

8.5.2 AI-Driven Operational Intelligence

Predictive Maintenance: The system should utilize current IoT data streams to train Machine Learning models (e.g., LSTMs) capable of predicting equipment failure and automating the creation of preventative Work Orders.

Anomaly Detection: Implementing real-time monitoring algorithms will allow the system to visually highlight "at-risk" areas in the 3D model based on dynamic sensor thresholds (e.g., overheating, fluid leakage, or excessive vibration).

8.5.3 Advanced GIS & Semantic Context

Knowledge Graphs (Knowledge-based Twin): Operational data evolution demands transitioning from a relational database to a Knowledge Graph (RDF/Neo4j). This would allow the system to natively understand semantic relationships (e.g., *this pipe provides water to these specific rooms*), enabling advanced cross-impact analysis during critical failures.

Regulatory Compliance (Automated Rule Checking): The development of an IFC-agnostic rule engine is necessary to automatically verify if the loaded model complies with local building codes, such as accessibility radii, fire safety protocols, and minimum clearance distances.

8.5.4 Immersion and Field Integration (WebXR)

Augmented Reality (AR): To bridge the gap between the web and fieldwork, the framework projects the implementation of WebXR support. This will overlay the BIM model onto physical construction sites with high precision, allowing field technicians to see "hidden" infrastructure (pipes and cables) inside walls or underground.

Remote Collaboration: Enabling multi-user sessions where stakeholders can join a shared VR/AR room to review the model, annotate, and discuss clashes in real-time.

8.5.5 Industry Standardization (BCF & ISO)

BCF (BIM Collaboration Format) Support: In alignment with the project's 100% open-source philosophy, the future architecture must integrate BCF within a strict openBIM workflow. This will allow all issues discovered in this web viewer (Clashes, Notes) to be exported and opened directly and transparently across any openBIM-compliant ecosystem, guaranteeing true interoperability without reliance on proprietary authoring software.

ISO 19650 Compliance: Finally, the system's Information Management flow must be structured to strictly follow international ISO 19650 standards for Digital Twins and BIM ifecycle management.

8.5.6 Regulatory Audit and Automatic Compliance

Development of a BIM Rule Engine based on semantic logic to automatically verify compliance with local building codes directly on the IFC model, alongside tolerance-sensitive analysis (Clearance Detection) for MEP coordination.

8.5.7 Operational Intelligence and Graph Data Fusion

Utilizing integrated sensor data for dynamic "What-if" scenario simulations to predict the impact of operational changes on thermal comfort. Additionally, transitioning from relational

databases to graph databases (e.g., Neo4j) to map complex relationships and enable cross-cutting impact queries across the semantic web.

8.5.8 Lifecycle and Collaboration (4D/5D)

Integrating native BIM Collaboration Format (BCF) support to allow transparent, bidirectional synchronization of issues and clashes with authoring tools, as well as incorporating the 4D dimension to detect logistic and temporal conflicts during project execution.

9. Summary

Overview & Problem Statement

This master's thesis by Jorge David Velazquez Cupido addresses the fragmentation of information in Facility Management (FM) during the Operation and Maintenance (O&M) phase of a building. Traditionally, FM teams rely on disconnected, proprietary platforms separating static Building Information Modeling (BIM) data, Geographic Information Systems (GIS) context, and live Internet of Things (IoT) sensor telemetry. This fragmentation creates data silos, relies on expensive software licenses, and prevents the effective use of BIM as a primary operational database.

Proposed Solution

To bridge this gap, the author develops a modular, open-source, web-based framework that integrates BIM, GIS, and IoT into a single Digital Twin environment accessible via a standard web browser. Guided by a Design Science Research methodology, the project delivers a functional Minimum Viable Product (MVP) that avoids vendor lock-in and allows asset owners to maintain total data sovereignty by supporting on-premise deployments.

Core Architecture & Technical Features

- **IFC Semantic Backbone:** The framework uses the open Industry Foundation Classes (IFC) standard, specifically leveraging the persistent GlobalId (GUID) as a universal primary key. This allows operational CRUD (Create, Read, Update, Delete) records, such as work orders and maintenance notes, to be reliably linked to specific 3D physical elements in a relational database.

- **Hybrid BIM-GIS Visualization:** The client-side interface orchestrates two overlapping rendering engines. It uses CesiumJS for global geospatial context (terrain and mapping) and Three.js accelerated by WebAssembly (WASM) for rendering highly detailed local BIM geometry. It also introduces a manual calibration workflow to align IFC models that lack native georeferencing coordinates.

- **Real-Time IoT Integration:** To handle dynamic building telemetry without overloading the browser, the system streams live sensor data (e.g., temperature, CO_2) using lightweight Server-Sent Events (SSE). This allows the interface to generate dynamic 3D volumetric heatmaps in near real-time without requiring full scene re-renders.

- **Decoupled Backend:** A Python-based FastAPI backend powered by IfcOpenShell manages heavy geometric processing, database persistence, and safe, server-side structural mutations of the IFC files. This strict separation ensures the frontend remains lightweight and the original file data is not corrupted.

Results & Conclusion

Validated through an HFT Stuttgart campus case study, the prototype successfully executes 23 operational FM workflows, including advanced clash detection, quantity takeoffs (QTO), and interactive BIM editing. The research demonstrates

that an entirely open-source ecosystem can effectively democratize Digital Twins, breaking down data silos and paving the way for predictive, data-driven facility management.

Figure 19 Summary

Appendixes

Appendix 1 User Workflows & Operational Procedures

IFC Viewer & Facility Management System

This appendix documents the operational workflows of the web-based system, including user-facing steps and the corresponding process diagrams. Diagram sources are provided in Mermaid syntax (as code listings) so they can be rendered as images if needed.

Workflow 1: Model Loading & Management

Objective: Load and manage multiple localized BIM models (Federation).

User Steps

1. **Launch System**: Open the web application. Initial view: Cesium 3D map.
2. **Load IFC File**:
   - Click the **"Open IFC"** button (Folder Icon).
   - Select a valid .ifc file (e.g., Architecture.ifc).
   - *System Response*: Parses geometry and adds it to the scene.
3. **Federation (Multi-Model)**:
   - Repeat step 2 to load additional files (e.g., MEP.ifc, Structure.ifc).
   - *System*: Overlays models in the same 3D space based on shared coordinates.
4. **Manage Loaded Models**:
   - Open **"Loaded Models"** panel.
   - Toggle visibility (Eye icon) for individual models.
   - Click "Unload" (Trash icon) to remove a model from memory.

Process Diagram

```
graph LR
  A[Start App] --> B[Load Architecture.ifc]
  B --> C[Load MEP.ifc]
  C --> D{Federated View}
  D --> E[Manage Visibility]
  D --> F[Unload Model]
```

Workflow 2: Model Tree Navigation

Objective: Explore the model's spatial hierarchy and isolate elements.

User Steps

1. **Open Tree**: Click the **"Tree"** icon in the toolbar.
2. **Navigate Hierarchy**:
   - Expand nodes: **Project** -> **Site** -> **Building** -> **Storey**.
   - *System*: Displays icon-coded elements (e.g., for Doors, for Windows).
3. **Search**:

- Type "Wall" in the search bar.
- *System*: Filters tree to show only matching elements.

4. **Isolate/Ghost**:
   - Click the **Ghost Icon ()** on a Storey node (e.g., "Level 2").
   - *System*: Makes all other storeys transparent, highlighting the selected level.

Process Diagram

```
graph TD
    A[Open Tree] --> B[Expand Level Node]
    B --> C{Action}
    C -->|Search| D[Filter Elements]
    C -->|Toggle| E[Hide/Show Node]
    C -->|Ghost| F[Isolate Level]
    F --> G[Transparent Context]
```

Workflow 3: 2D Plan Navigation

Objective: Navigate the building using generated 2D floor plans.

User Steps

1. **Access Plans**:
   - Click the **"2D Plans"** icon in the sidebar (or map icon in Tree).
2. **Select Level**:
   - Choose a specific storey (e.g., "Level 1").
   - *System Response*:
   - Cuts 3D model horizontally at storey height.
   - Switches camera to **Top-Down Orthographic**.
   - Displays **Plan Tools** panel with QTO stats.
3. **Review Stats**:
   - Check element counts (e.g., "25 Walls", "14 Doors").
4. **Exit Plan View**:
   - Click **"Exit Plan View"** to restore 3D perspective.

Process Diagram

```
sequenceDiagram
    participant User
    participant Viewer
    participant Camera

    User->>Viewer: Select "Level 1"
    Viewer->>Viewer: Clip Model @ Z-Height
    Viewer->>Camera: Switch to Orthographic
    Viewer-->>User: Show 2D Plan + Stats
    User->>Viewer: Exit View
    Viewer->>Camera: Restore Perspective
```

Workflow 4: Measurements & Dimensioning

Objective: Perform precise geometric measurements.

User Steps

1. **Activate Tool**: Click the **"Ruler"** icon.
2. **Snap to Geometry**: Hover over edges; cursor snaps to vertices (red dot).
3. **Measure**:
   - Click **Start Point** -> Move -> Click **End Point**.
   - *Result*: Dimension line rendered with distance label (e.g., 3.50m).
4. **Area (Polygon)**:
   - Click **"Medir Área"**.
   - Click 3+ points to define a polygon.
   - Double-click to close.
   - *Result*: Translucent fill with Area label (e.g., 45.2 m²).
5. **Clear**: Click "Delete All" to remove lines.

Process Diagram

```
graph LR
   A[Activate Ruler] --> B[Hover Geometry]
   B --> C{Snap Vertex?}
   C -- Yes --> D[Click Start]
   D --> E[Click End]
   E --> F[Render Dimension]
```

Workflow 5: Calibration & Georeferencing

Objective: Align BIM model with real-world terrain (Cesium).

User Steps

1. **Open Calibration**: Click **"Geo Calibration"** sidebar menu.
2. **Manual Adjustment**:
   - Adjust **Lat/Lon** offset inputs.
   - Use **Rotation** slider to align orientation.
3. **Snap to Terrain**:
   - Click **"Snap to Terrain"**.
   - *System*: Queries Cesium terrain API for elevation at model centroid.
   - *System*: Automatically updates model Z-height to sit on ground.
4. **Mask Terrain**:
   - Click **"Mask Terrain"** to hide satellite map under the building foundation.
5. **Click-to-Place (Anchor)**:
   - Click **"Fijar Ancla"** to enable map interaction.
   - Click anywhere on the 2D map to instantly relocate the model to that point.

Process Diagram

```
graph TD
   A[Calibration Panel] --> B{Adjustment Type}
   B -->|Manual| C[Update Lat/Lon/Rot]
   B -->|Auto| D[Snap to Terrain]
   D --> E[Query Elevation API]
   E --> F[Update Model Height]
```

Workflow 6: BIM Editing (Transformations)

Objective: Correct element placement errors and save changes.

User Steps

1. **Enter Editor**: Click **"BIM Editor"** (Hammer icon).
2. **Select Element**: Click target object (e.g., misplaced wall).
   - *System*: Isolates element in yellow visual subset with Gizmo.
3. **Transform**:
   - Drag axes (Red X, Blue Z) or rotate.
   - *Visual*: Object moves in real-time.
4. **Commit**:
   - Click **"Save"**.
   - *Backend*: Calculates Delta, applies Unit Scaling (mm/m), updates source IFC file.
5. **Persistence**: Page reloads; element is permanently moved.

Process Diagram

```
sequenceDiagram
    participant User
    participant Editor
    participant Backend

    User->>Editor: Select Element
    Editor->>Editor: Create Visual Subset
    User->>Editor: Move Element (Delta)
    User->>Editor: Click Save
    Editor->>Backend: POST /transform {delta, scale}
    Backend->>Backend: Apply Matrix to IFC
    Backend-->>Editor: Success
```

Workflow 7: Asset Management (New!)

Objective: Place external objects (Furniture, Equipment) into the scene.

User Steps

1. **Open Assets**: Click **"Asset Editing"** sidebar menu.
2. **Select Type**: Choose "Table", "Chair", or "Equipment" from dropdown.
3. **Place**:
   - Click **"Enable Creator"**.
   - Click anywhere on a floor/surface in 3D.
   - *System*: Instantiates a GLB model (or proxy box) at that location.
4. **Move/Edit**:
   - Click an existing asset to select it (Yellow Wireframe).
   - Drag to reposition.
5. **Upload Custom**:
   - Upload a .glb file to add new types to the library.

Process Diagram

```
graph LR
  A[Select Asset Type] --> B[Enable Creator]
  B --> C[Raycast Surface]
  C --> D[Instantiate GLB]
  D --> E[Save to DB]
```

Workflow 8: Systems & Disciplines (Tech Mode)

Objective: Visualize specific building systems (MEP, Electrical, Structural).

User Steps

1. **Toggle Tech Mode**: Click **"Tech Mode"** (or Disciplines menu).
2. **Visual filtering**:
   - *System*: Hides architectural skin (Walls transparent/hidden).
   - **MEP**: Shows Pipes/Ducts in **Green**.
   - **Electrical**: Shows Cables/Boxes in **Yellow**.
   - **Structural**: Shows Beams/Columns in **Grey**.
3. **Context**: Allows seeing internal systems without wall occlusion.

Process Diagram

```
graph TD
  A[Toggle Tech Mode] --> B[Hide Architecture]
  B --> C[Create Subsets]
  C --> D[MEP (Green)]
  C --> E[Elec (Yellow)]
  C --> F[Struct (Grey)]
```

Workflow 9: Clash Detection

Objective: Identify physical conflicts between models (e.g., Pipe vs Beam).

User Steps

1. **Open Clash Manager**: Click **"Clash Detection"** menu.
2. **Run Check**:
   - Click **"Run Detection"**.
   - *System*: Analyzes bounding box intersections between loaded models.
3. **Review Conflicts**:
   - List of clashes appears.
   - *Visual*: Red sphere/box highlights the conflict area in 3D.
4. **Export**:
   - Click **"Export CSV"** to download a report of all conflicts.

Process Diagram

```
graph TD
  A[Start Detection] --> B[Iterate Models]
  B --> C{Intersect BBox?}
  C -- Yes --> D[Register Conflict]
```

```
    D --> E[Highlight in 3D]
    E --> F[Generate Report]
    C -- No --> G[Ignore]
```

Workflow 10: Model Comparison (Diff)

Objective: visualize changes between two versions of a model.

User Steps

1. **Load Models**: Load "Version A" (Old) and "Version B" (New).
2. **Run Diff**:
    - Click **"Compare Models"**.
3. **Visual Results**:
    - **Green**: Added elements (New in B).
    - **Red**: Removed elements (Deleted from A).
    - **Grey (Transparent)**: Unchanged context.

Process Diagram

```
graph LR
    A[Compare A vs B] --> B{Check IDs}
    B -->|In B only| C[Green (Added)]
    B -->|In A only| D[Red (Removed)]
    B -->|In Both| E[Grey (Unchanged)]
```

Workflow 11: IoT Sensor Monitoring

Objective: Visualize real-time sensor data (Temp, CO2).

User Steps

1. **Toggle IoT**: Activate **"IoT Sensors"**.
    - *Visual*: Spheres appear in rooms.
2. **Status Colors**:
    - **Green** (Normal), **Red** (Hot/Danger), **Blue** (Cold).
    - *Animation*: Alerting sensors pulse.
3. **Heatmap Mode**:
    - Toggle **"Heatmap"**.
    - *System*: Colors entire room volume based on average sensor data.

Process Diagram

```
sequenceDiagram
    participant Frontend
    participant Stream

    Frontend->>Frontend: Enable IoT
    Frontend->>Stream: Connect SSE
    loop Stream
        Stream-->>Frontend: Update {id, val}
        Frontend->>Frontend: Update Color/Pulse
    end
```

Workflow 12: Maintenance Work Orders

Objective: Manage maintenance tasks linked to elements.

User Steps

1. **Select Element**: Click a target object (e.g., AC Unit).
2. **Open Work Orders**: Sidebar section "Work Orders".
3. **Create Order**:
    - Title: "Replace Filter".
    - Priority: "High".
    - Click **"Create"**.
4. **Tracking**: Order is linked to IFC GUID. Status badges track progress.

Process Diagram

```
graph LR
    A[Select Element] --> B[Fill Form]
    B --> C[Create Order]
    C --> D[Link to GUID]
    D --> E[Show in List]
```

Workflow 13: Reports (QTO)

Objective: Generate Quantity Take-Off reports.

User Steps

1. **Open Reports**: Click **"Reports"** toolbar icon.
2. **Review Data**:
    - Table shows counts/areas by Category (Walls, Slabs, etc.).
3. **Export**:
    - Click **"Download JSON/CSV"**.
    - *System*: Generates a file summary of the current model's inventory.

Process Diagram

```
graph TD
    A[Scan Model] --> B[Aggregate by Type]
    B --> C[Compute Counts]
    C --> D[Render Table]
    D --> E[Export CSV]
```

Workflow 14: Section Box & Clipping

Objective: Slice the model to reveal internal details using 6 clipping planes.

User Steps

1. **Activate Tool**: Click the **"Section Box" (Cortes)** icon in the toolbar.
    - *System*: Displays a translucent box around the model's bounds with interactive handles.

2. **Clip Model**:
    - Grab any of the 6 colored face handles (Red for X, Green for Y, Blue for Z).
    - Drag the handle inwards to slice the geometry.
    - *Visual*: The model geometry is clipped in real-time.
3. **Toggle Helper**:
    - Click **"Toggle Box"** to hide the manipulator while keeping the cut active.
4. **Reset**:
    - Click **"Reset"** to restore the box to the model's full bounding box.

Process Diagram

```
graph LR
    A[Activate Section Box] --> B[Show Gizmo]
    B --> C[Drag Plane Handle]
    C --> D[Update Clip Planes]
    D --> E[Render Cut View]
    E --> F{Done?}
    F -- Yes --> G[Hide Gizmo]
    F -- No --> C
```

Workflow 15: ViewCube Navigation

Objective: Quickly switch between standard camera views (Top, Front, Right).

User Steps

1. **Locate ViewCube**: Find the interactive 3D cube in the top-right corner.
2. **Switch View**:
    - Click a face (e.g., **"TOP"**, **"FRONT"**).
    - *System*: Smoothly animates the camera to align with that axis.
3. **Rotate View**:
    - Click the **Arrows** around the cube to rotate 90° in that direction.
4. **Sync**:
    - The ViewCube rotates in real-time as you manually orbit the main model.

Process Diagram

```
graph TD
    A[User Action] --> B{Interaction}
    B -->|Click Face| C[Animate to Ortho View]
    B -->|Click Arrow| D[Rotate 90°]
    B -->|Orbit Main| E[Sync Cube Rotation]
```

Workflow 16: GIS Context (Surroundings)

Objective: Load and visualize surrounding buildings using GIS data.

User Steps

1. **Configure Location**: Ensure simple Lat/Lon is set in Calibration.
2. **Load Context**:
    - Click **"Load Surroundings"** or **"Context"**.

- *System*: Fetches building footprints from GIS API (/gis/footprint).

3. **Visuals**:
   - Surrounding buildings appear as extruded grey masses.
   - Terrain is flattened or meshed to match.
4. **Hide/Show**:
   - Toggle **"Show Context"** to focus on your specific project vs. environment.

Process Diagram

```
sequenceDiagram
    participant User
    participant ContextMgr
    participant GIS_API

    User->>ContextMgr: Load Context
    ContextMgr->>GIS_API: GET /footprint?lat,lon
    GIS_API-->>ContextMgr: GeoJSON Polygons
    ContextMgr->>ContextMgr: Extrude Shapes
    ContextMgr-->>User: Render City Model
```

Workflow 17: Box Selection

Objective: Select multiple elements at once using a drag-box.

User Steps

1. **Activate Tool**: Hold **Shift** + Click Drag (or activate specific tool if button exists).
2. **Draw Box**:
   - Click and drag a rectangle on screen.
   - *Visual*: Blue translucent selection rectangle appears.
3. **Selection**:
   - Release mouse button.
   - *System*: Selects all items whose center/geometry falls within the screen rectangle.
   - *Visual*: All captured items highlight in yellow.

Process Diagram

```
graph LR
    A[Start Drag] --> B[Draw Rect (Screen)]
    B --> C[Release]
    C --> D[Project Frustum]
    D --> E[Find Intersects]
    E --> F[Select All]
```

Workflow 18: Advanced Search & Filtering (Explorer)

Objective: Find elements by textual search or filter by specialized IFC Types.

User Steps

1. **Access Explorer**: Click **"IFC Explorer"** sidebar (Magnifying Glass).
2. **Switch to Search**: Click the **"Búsqueda"** tab.
3. **Configure Criteria**:

- **Text**: Enter a name or ID (e.g., "Muro", "2531").
- **Type**: Select element category (e.g., IFCCOLUMN, IFCDOOR).

4. **Execute**:
   - Click **"Buscar"**.
   - *System*: Queries the model index.
5. **Interact with Results**:
   - Click a result in the list to **Select & Zoom** to that element.
   - Click **"Select All"** to highlight all matching items.

Process Diagram

```
graph TD
  A[Open Explorer] --> B[Set Filters]
  B --> C[Click Search]
  C --> D[Query Index]
  D --> E[Show Results List]
  E --> F{User Action}
  F -->|Click Item| G[Zoom to Element]
  F -->|Select All| H[Highlight Batch]
```

Workflow 19: Element Annotations (Notes)

Objective: Attach text notes or maintenance info to specific IFC elements.

User Steps

1. **Select Element**: Click on an object in the 3D view (e.g., a damaged Wall).
2. **Open Notes**: In the sidebar, scroll to the **"Notas / Comentarios"** section.
3. **Add Note**:
   - Type text in the input field: "Check crack width".
   - Click **"Agregar Nota"** (Send button).
   - *System*: Saves note to backend linked to GlobalID.
4. **Review**:
   - Notes appear in a list below the form with timestamp and author.
   - Click **"X"** to delete a note.

Process Diagram

```
graph TD
  A[Select Element] --> B[Type Note]
  B --> C[Submit]
  C --> D[POST /api/notes]
  D --> E[Update List]
  E --> F[Show Timestamp]
```

Workflow 20: Language Switching

Objective: Change the interface language (Internationalization).

User Steps

1. **Locate Selector**: Top-right corner, flag icon (e.g., ES).

2. **Open Menu**: Click the flag button to reveal options.
3. **Select Language**:
    - Choose **English**, **Español**, or **Deutsch**.
    - *System*: Instantly updates all UI text labels without reloading.

Process Diagram

```
graph LR
    A[Click Flag] --> B[Show List]
    B --> C[Select Language]
    C --> D[Update DOM Text]
    D --> E[Update Placeholders]
```

Workflow 21: Visualization Control

Objective: Adjust rendering style, lighting, and transparency for better analysis.

User Steps

1. **Locate Controls**: Sidebar > "Visualization" section (or press keys 1-4).
2. **Render Mode**:
    - **Realistic**: Standard material view.
    - **Wireframe**: Lines only (Key 2).
    - **X-Ray**: Semi-transparent (Key 4).
3. **Lighting**:
    - Adjust **Ambient Light** slider to brighten shadows.
    - Adjust **Directional Light** slider for contrast.
4. **Background**: Change sky color (White, Black, Gradient).

Process Diagram

```
graph TD
    A[User Changes Setting] --> B{Type?}
    B -->|Mode| C[Set Material]
    B -->|Light| D[Update Intensity]
    B -->|Color| E[Set ClearColor]
    C --> F[Re-render Scene]
```

Workflow 22: GIM Layer (Geo-Footprint)

Objective: Toggle 2D GIS footprint overly on the map context.

User Steps

1. **Locate Toggle**: Click **"Toggle GIM"** button in top bar or map overlay.
2. **Action**:
    - *System*: Fetches footprint.geojson from backend.
    - *System*: Projects coordinates to Leaflet map layer.

3. **Visual**:
    - Blue outline appears on the 2D map showing the building footprint.
    - Click to hide.

Process Diagram

```
graph LR
    A[Click Toggle GIM] --> B{Visible?}
    B -->|No| C[Fetch GeoJSON]
    C --> D[Add Leaflet Layer]
    B -->|Yes| E[Remove Layer]
```

Workflow 23: Project Creation (Upload)

Objective: Create a new persistent project by uploading an IFC file to the server.

User Steps

1. **Initiate**: Click **"Nuevo Proy."** button (Top Bar).
2. **Select File**: Choose a .ifc file from local storage.
3. **Upload**:
    - *System*: Uploads file to POST /sites/upload.
    - *System*: Analyzes file and creates a new Project ID.
4. **Completion**:
    - System alerts "Proyecto creado".
    - The new project is automatically selected and loaded.

Process Diagram

```
graph TD
    A[Click New Project] --> B[Pick File]
    B --> C[POST /sites/upload]
    C --> D{Success?}
    D -->|Yes| E[Create Site ID]
    E --> F[Auto-Load Model]
    D -->|No| G[Show Error]
```

Appendix 2 Project Architecture

Backend (FastAPI + IfcOpenShell) and Frontend (Vite + Three.js + Cesium)

This appendix summarizes the system architecture of the thesis prototype, including file structure, component diagrams, critical data flows, and the FastAPI endpoint catalog.

Project Architecture

File Structure

Simplified view of the project (Backend + Frontend).

```
Thesis/
├── ifc-api/                    # Backend (FastAPI + IfcOpenShell)
│   ├── app/
│   │   ├── core/               # Configuration (config.py)
│   │   ├── models/             # Pydantic/SQLModel Models
│   │   │   ├── asset.py
│   │   │   ├── editor.py
│   │   │   ├── iot_models.py
│   │   │   ├── logistics.py
│   │   │   ├── schemas.py
│   │   │   └── work_order.py
│   │   ├── routers/            # API Endpoints
│   │   │   ├── assets.py
│   │   │   ├── editor.py
│   │   │   ├── gis.py
│   │   │   ├── hypermedia.py
│   │   │   ├── ifc.py
│   │   │   ├── ifc_editor.py     # IFC Editor (Transformations)
│   │   │   ├── iot.py
│   │   │   ├── logistics.py
│   │   │   ├── sites.py
│   │   │   └── work_orders.py
│   │   ├── services/           # Business Logic
│   │   │   ├── data_service.py
│   │   │   ├── ifc_export.py
│   │   │   ├── ifc_modifier.py
│   │   │   ├── ifc_service.py
│   │   │   ├── iot_service.py
│   │   │   └── logistics.py
│   │   ├── database.py         # DB Connection (SQLite)
│   │   └── main.py             # Main FastAPI Entry point
│   ├── data/                   # Persistent Data (.json, .db)
│   ├── ifc_files/              # Uploaded IFC Files
│   ├── static/                 # Static Files
│   └── requirements.txt
│
└── ifc-viewer/                 # Frontend (Vite + Vue/Vanilla JS + Three.js)
    ├── public/                 # Public Assets (wasm, models)
    ├── src/
    │   ├── components/         # UI/Logic Components
    │   │   ├── AssetManager.js
    │   │   ├── BimEditor.js      # 3D Editor Logic
    │   │   ├── BoxSelectionManager.js
    │   │   ├── CesiumMap.js      # GIS Map
    │   │   ├── ClippingManager.js
    │   │   ├── ContextManager.js # 3D Environment (Surroundings)
    │   │   ├── EditorToolbar.js
    │   │   ├── Measurements.js
    │   │   ├── NotesManager.js
    │   │   ├── PlansManager.js
    │   │   ├── SearchManager.js
    │   │   ├── SelectionManager.js
    │   │   ├── SystemsManager.js # Tech Mode
    │   │   ├── VisualizationControls.js
    │   │   └── ...
    │   ├── styles.css          # Global Styles
    │   └── main.js             # Application Entry Point
    ├── index.html              # Main HTML
```

```
    ├── package.json
    └── vite.config.js
```

Table 3 Project Architecture

Component Diagrams

Frontend Architecture (Client)

The client is a complex SPA that orchestrates two rendering engines (Three.js + Cesium) and multiple state managers.

```
graph TD
    User[User] --> Main[main.js (Entry Point)]

    subgraph "Graphics Engines"
        Main --> Three[Three.js (BIM / Local)]
        Main --> Cesium[CesiumJS (Global Context)]
        Cesium -.->|Sync Camera| Three
    end

    subgraph "Managers"
        Main --> IFC[IFC Manager (web-ifc)]
        Main --> BIM[BimEditor (Transform)]
        Main --> GIS[ContextManager (Surroundings)]
        Main --> IoT[IoT Manager (SSE)]
        Main --> Assets[AssetManager]
        Main -->  Plans[PlansManager]
    end

    subgraph "UI Overhead"
        Main --> UIM[UIManager]
        Main --> VC[Vis. Controls]
        Main --> Cube[ViewCube]
    end
```

Table 4 Frontend Architecture

Backend Architecture (Server)

The FastAPI server acts as a data orchestrator and geometric processor.

```
graph TD
    Client[Frontend API] -->|HTTP/REST| API[FastAPI Router]

    subgraph "Service Layer"
        API --> S_IFC[IFC Service]
        API --> S_IOT[IoT Service]
        API --> S_GIS[GIS Service]
    end

    subgraph "Processing"
        S_IFC --> IOS[IfcOpenShell (Python)]
        S_GIS --> OSm[Overpass API / GeoJSON]
    end

    subgraph "Persistence"
        S_IFC --> DB[(SQLite / PostgreSQL)]
        S_IOT --> DB
        IOS --> Disk[File System (.ifc)]
    end
```

Table 5 Backend Architecture

Critical Data Flows

1. Element Transformation (BIM Editor)

The most complex flow requiring coordinate translation.

1. **Frontend:** TransformControls moves an object in Three.js (Axis: Y-Up).
2. **Calculation:** The Delta (Difference) between original and new position is calculated.
3. **Sending:** POST /transform sends { delta: [x,y,z], globalId }.
4. **Backend:** IfcOpenShell opens the IFC file.
5. **Mapping:** Converts Y-Up coordinates (Three) to Z-Up (IFC).
6. **Persistence:** Writes the new IfcLocalPlacement matrix and saves the file.

2. IoT Synchronization (Real-Time)

High-frequency stream handling.

1. **Backend:** Data generator simulates readings every 5s and pushes them to the SSE channel.
2. **Transport:** Server-Sent Events maintains an open unidirectional connection.
3. **Frontend:** EventSource receives the onmessage event.
4. **Shader:** Three.js updates the color uniform in the specific IfcSpace material without re-rendering the entire geometry.

3. GIS Fusion (Coordinates)

1. **Cesium:** Provides the WGS84 (Lat/Lon) reference frame.
2. **Three.js:** Operates in Local Cartesian coordinates (0,0,0 at site center).
3. **Bridge:** The transformation matrix is updated every frame to ensure Three.Camera "follows" Cesium.Camera, creating the illusion of a single world.

API Endpoints (FastAPI)

Complete list of available endpoints in the Backend.

General

Method	Endpoint	Description
GET	/	Root (Welcome)
GET	/health	Health Check (GIS)

IFC Viewer & Data

Method	Endpoint	Description
GET	/sites	List available sites/projects
GET	/sites/{site_id}	Site details
POST	/sites/upload	Upload new IFC file
GET	/ifc/files/{filename}	Download IFC file
GET	/ifc/properties/by-express-id/{express_id}	Element properties
GET	/ifc/element/{site_id}/{global_id}	Find element by GUID
GET	/ifc/search/{site_id}	Search elements by text/type
GET	/ifc/types/{site_id}	List IFC types in model
GET	/ifc/storeys/{site_id}	List levels/storeys
GET	/ifc/qto/{site_id}	Global Quantities Take-Off (QTO)
GET	/ifc/qto_by_storey/{site_id}	QTO by storey
GET	/ifc/georef/{site_id}	Georeference information
POST	/ifc/export/{site_id}	Export IFC with injected assets

BIM Editor (Transformations)

Method	Endpoint	Description
POST	/api/elements/{global_id}/transform	**Move/Rotate/Scale element**
POST	/api/elements/create	Create new element
GET	/api/elements/health	Editor Health Check

POST	/projects/{project_id}/deltas	Save change (delta)
GET	/projects/{project_id}/deltas	List pending changes
POST	/projects/{project_id}/commit	Apply changes to IFC file

GIS & Maps

Method	Endpoint	Description
GET	/gis/info	Map center info
GET	/gis/footprint	Get building footprints (OSM)
GET	/gim/{site}/footprint	Site specific footprint

IoT (SensorThings API)

Method	Endpoint	Description
GET	/iot/Things	List "Things" (Sensors)
GET	/iot/Things('{thing_id}')/Datastreams	Datastreams of a Thing
GET	/iot/Datastreams('{ds_id}')/Observations	Observations (Readings)
GET	/iot/Things('{thing_id}')/HistoricalLocations	Location history
POST	/iot/import	Import sensor CSV
GET	/iot/latest/{site_id}/{express_id}	Latest reading (Mock)
GET	/iot/sensors	GeoJSON of all sensors

Work Orders (Facility Management)

Method	Endpoint	Description
GET	/work-orders/	List orders
POST	/work-orders/	Create order
GET	/work-orders/{id}	View order
PATCH	/work-orders/{id}	Update order
DELETE	/work-orders/{id}	Delete order

Asset Management

Method	Endpoint	Description
GET	/assets/	List assets (Furniture, etc.)
POST	/assets/	Create asset in DB
POST	/assets/upload	Upload 3D model (.glb)
PUT	/assets/{id}	Update asset
DELETE	/assets/{id}	Delete asset

Notes & Overrides

Method	Endpoint	Description
GET	/ifc/notes/{site_id}/{global_id}	View element notes
POST	/ifc/notes/{site_id}/{global_id}	Create note
GET	/ifc/overrides/{site_id}/{express_id}	View overrides (colors, etc.)
PUT	/ifc/overrides/{site_id}/{express_id}	Save override

Hypermedia (HATEOAS)

Method	Endpoint	Description
GET	/hm	Root Hypermedia
GET	/hm/sites	Sites with navigable links
GET	/hm/sites/{site_id}	Site detail with links

Logistics

Method	Endpoint	Description
POST	/logistics/calculate	Calculate optimal route (Milk Run)

Overview of Tools Used

Activity	**Description**	**Used AI tools** (if applicable)[1]	**Description of the use of AI tools** (Type of use, affected sections of the work, etc.)
Idea generation & conceptualiz ation	– Development of ideas, research goals, objectives and research questions – Identification and definition of relevant concepts		
Literature research and analysis	– Search for relevant literature – Review of potentially relevant literature – Summaries of relevant literature	NotebookLM (“Google NotebookLM \textbar AI Research Tool & Thinking Partner,” n.d.)	Help us to create summaries.
Methodology	– Search for a suitable methodology – Development and adaptation of the methodology to the research question(s)		
Coding	– Creating and documenting code, algorithms, and software – Testing and debugging existing code, algorithms, and software – Understanding of existing code, algorithms, software	Antigravity (*Google Antigravity*, n.d.) ChatGPT (“ChatGPT,” n.d.)	Tool that helps to create code.
Data collection and analysis	– Collection of primary or secondary data – Qualitative data analysis (including summaries and coding) – Quantitative data analysis (including statistical evaluation) – Mathematical, computer-aided, or		

List the names of the AI tools used, ChatGPT, Gemini, Microsoft 365, Grammarly, Translator, Google Translate, GitHub Copilot, Antigravity, Google Scholar.

	other formal techniques for modelling, simulation, and analysis		
Interpretation and validation	– Interpretation of the results – Derivation of implications for research and practice		
Text generation	– Generation of text on various topics in different sections of the work (including titles and summaries)		
Generation of tables and figures	– Generation of tables, diagrams, graphs, etc. on various topics or presentation of results in different sections of the work.	Napkin.ai (“Napkin AI - The Visual AI for Business Storytelling,” n.d.)	
Text review & editing	– Critical review, feedback, or revision of the content, organization, or grammar of the work – Proofreading – Reformulation or paraphrasing of text	Gemini (“Google Gemini,” n.d.)	
Source management	– Creating the reference list – Formatting of references	Zotero (*Zotero \textbar Your Personal Research Assistant*, n.d.)	
Other activities			

References

Abdelalim, A. M., Essawy, A., Alnaser, A. A., Shibeika, A., & Sherif, A. (2024). Digital Trio: Integration of BIM–EIR–IoT for Facilities Management of Mega Construction Projects. *Sustainability*, *16*(15), 6348. https://doi.org/10.3390/su16156348

Auer, M., Agugiaro, G., Billen, N., Loos, L., & Zipf, A. (2014). Web-based Visualization and Query of semantically segmented multiresolution 3D Models in the Field of Cultural Heritage. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, *II–5*, 33–39. ISPRS Technical Commission V Symposium (Volume II-5) - 23–25 June 2014, Riva del Garda, Italy. https://doi.org/10.5194/isprsannals-II-5-33-2014

Azari, P., Li, S., Shaker, A., & Sattar, S. (2025). Georeferencing Building Information Models for BIM/GIS Integration: A Review of Methods and Tools. *ISPRS International Journal of Geo-Information*, *14*(5). https://doi.org/10.3390/ijgi14050180

Barrile, V., La Foresta, F., Calcagno, S., & Genovese, E. (2024). Innovative System for BIM/GIS Integration in the Context of Urban Sustainability. *Applied Sciences*, *14*(19), 8704. https://doi.org/10.3390/app14198704

Butler, H., Daly, M., Doyle, A., Gillies, S., Schaub, T., & Hagen, S. (2016). *The GeoJSON Format* (Request for Comments RFC 7946). Internet Engineering Task Force. https://doi.org/10.17487/RFC7946

ChatGPT. (n.d.). In *ChatGPT*. Retrieved February 27, 2026, from https://chatgpt.com/

Chatsuwan, M., Ichinose, M., & Alkhalaf, H. (2025). Enhancing Facility Management with a BIM and IoT Integration Tool and Framework in an Open Standard Environment. *Buildings*, *15*(11), 1928. https://doi.org/10.3390/buildings15111928

FastAPI. (n.d.). Retrieved February 6, 2026, from https://fastapi.tiangolo.com/

Google Antigravity. (n.d.). Google Antigravity. Retrieved February 27, 2026, from https://antigravity.google/

Google Gemini. (n.d.). In *Gemini*. Retrieved February 27, 2026, from https://gemini.google.com

Google NotebookLM \textbar AI Research Tool & Thinking Partner. (n.d.). In *Google NotebookLM*. Retrieved February 27, 2026, from https://notebooklm.google/

Hakimi, O., Liu, H., & Abudayyeh, O. (2024). Digital twin-enabled smart facility management: A bibliometric review. *Frontiers of Engineering Management*, *11*(1), 32–49. https://doi.org/10.1007/s42524-023-0254-4

Herlé, S., & Blankenbach, J. (2024). Hypermedia-driven RESTful API for digital twins of the built environment. *Automation in Construction*, *165*, 105551. https://doi.org/10.1016/j.autcon.2024.105551

IfcOpenShell. (2026). *IfcOpenShell/IfcOpenShell* [C++]. https://github.com/IfcOpenShell/IfcOpenShell (Original work published 2015)

Industry Foundation Classes (IFC). (n.d.). *buildingSMART Technical*. Retrieved February 6, 2026, from https://technical.buildingsmart.org/standards/ifc/

ISO 41001:2018(en), Facility management—Management systems—Requirements with guidance for use. (n.d.). Retrieved February 4, 2026, from https://www.iso.org/obp/ui/en/#iso:std:iso:41001:ed-1:v1:en

Liu, H., Abudayyeh, O., & Liou, W. (2020). BIM-Based Smart Facility Management: A Review of Present Research Status, Challenges, and Future Needs. *Construction Research Congress 2020*, 1087–1095. https://doi.org/10.1061/9780784482865.115

Marmo, R., Polverino, F., Nicolella, M., & Tibaut, A. (2020). Building performance and maintenance information model based on IFC schema. *Automation in Construction*, *118*, 103275. https://doi.org/10.1016/j.autcon.2020.103275

Matarneh, S., Elghaish, F., Rahimian, F. P., Dawood, N., & Edwards, D. (2022). Automated and interconnected facility management system: An open IFC cloud-based BIM solution. *Automation in Construction*, *143*, 104569. https://doi.org/10.1016/j.autcon.2022.104569

Napkin AI - The visual AI for business storytelling. (n.d.). In *Napkin AI*. Retrieved February 27, 2026, from https://www.napkin.ai

Nguyen, T. D. H. N., Ly, D. H., Jang, H., Dinh, H. N. N., & Ahn, Y. (2025). Digital twin framework to enhance facility management for relocatable modular buildings. *Automation in Construction*, *176*, 106249. https://doi.org/10.1016/j.autcon.2025.106249

Noardo, F., Harrie, L., Arroyo Ohori, K., Biljecki, F., Ellul, C., Krijnen, T., Eriksson, H., Guler, D., Hintz, D., Jadidi, M., Pla, M., Sanchez, S., Soini, V.-P., Stouffs, R., Tekavec, J., & Stoter, J. (2020). Tools for BIM-GIS Integration (IFC Georeferencing and Conversions): Results from the GeoBIM Benchmark 2019. *ISPRS International Journal of Geo-Information*, *9*(9), 502. https://doi.org/10.3390/ijgi9090502

OGC SensorThings API Documentation | SensorUp OGC SensorThings API Developer Centre. (n.d.). Retrieved February 5, 2026, from https://developers.sensorup.com/docs/#historicalLocations_patch

OpenAPI Initiative – The OpenAPI Initiative provides an open source, technical community, within which industry participants may easily contribute to building a vendor-neutral, portable and an open specification for providing technical metadata for REST APIs – the "OpenAPI Specification" (OAS). (n.d.). Retrieved February 6, 2026, from https://www.openapis.org/

openBIM: Colaboración sin barreras en la construcción. (n.d.). buildingSMART Spain. Retrieved February 17, 2026, from http://www.buildingsmart.es/bim/openbim/

Otranto, R. B., Miceli Junior, G., & Pellanda, P. C. (2025). BIM-FM integration through openBIM: Solutions for interoperability towards efficient operations. *Journal of Information Technology in Construction*, *30*, 298–318. https://doi.org/10.36680/j.itcon.2025.012

Parsanezhad, P., & Dimyadi, J. (2014). *Effective facility management and operations via a BIM-based integrated information system*.

Pinti, L., Codinhoto, R., & Bonelli, S. (2022a). A Review of Building Information Modelling (BIM) for Facility Management (FM): Implementation in Public Organisations. *Applied Sciences*, *12*(3), 1540. https://doi.org/10.3390/app12031540

Pinti, L., Codinhoto, R., & Bonelli, S. (2022b). A Review of Building Information Modelling (BIM) for Facility Management (FM): Implementation in Public Organisations. *Applied Sciences*, *12*(3), 1540. https://doi.org/10.3390/app12031540

Savalskii, D. (2025). *Integrating Building Information Modeling (BIM) and Geographic Information Systems (GIS) for Enhanced Management and Efficiency of Urban Infrastructure in Smart Cities*.

Shi, Y. (2025). Use of Digital Twin Technologies in Facility Management and Related Industries: A Bibliometric Review. *Academic Journal of Science and Technology*, *14*(3), 91–103. https://doi.org/10.54097/8vggtq23

Stojanovic, V. (2021). *Digitale Zwillinge für gebaute InnenumgebungenDigital twins for indoor built environments* (p. 33163 KB, xxiii, 181 pages) [Universität Potsdam; Application/pdf]. https://doi.org/10.25932/PUBLISHUP-50913

That Open. (2026). *ThatOpen/web-ifc-three* [JavaScript]. https://github.com/ThatOpen/web-ifc-three (Original work published 2021)

That Open. (2026). *ThatOpen/engine_web-ifc* [TypeScript]. https://github.com/ThatOpen/engine_web-ifc (Original work published 2020)

Volk, R., Stengel, J., & Schultmann, F. (2014). Building Information Modeling (BIM) for existing buildings—Literature review and future needs. *Automation in Construction*, *38*, 109–127. https://doi.org/10.1016/j.autcon.2013.10.023

Wang, N., Cheng, R., Chen, J., & Zhang, M. (2024). Streamlining BIM models for GIS integration: An advanced approach to building envelope extraction. *International Journal of Digital Earth*, *17*(1), 2368708. https://doi.org/10.1080/17538947.2024.2368708

Wijeratne, P. U., Gunarathna, C., Yang, R. J., Wu, P., Hampson, K., & Shemery, A. (2024). BIM enabler for facilities management: A review of 33 cases. *International Journal of Construction Management*, *24*(3), 251–260. https://doi.org/10.1080/15623599.2023.2222962

Wills, N., & Bartels, N. (2022). On the Applicability of Open Standard Exchange Formats for Demand-Oriented Facility Management (FM) Service Delivery in the Context of the Cross-Lifecycle Building Information Modeling (BIM) Method. *Facility Management*.

Zotero \textbar Your personal research assistant. (n.d.). Retrieved February 27, 2026, from https://www.zotero.org/